

[Open in app](#)[Sign up](#)[Sign in](#)

Medium



CI/CD Project: Deploy a 3-tier Microservice Voting App using ArgoCD and Azure DevOps Pipeline

23 min read · Sep 6, 2024



GABRIEL OKOM

[Follow](#)

Listen



Share

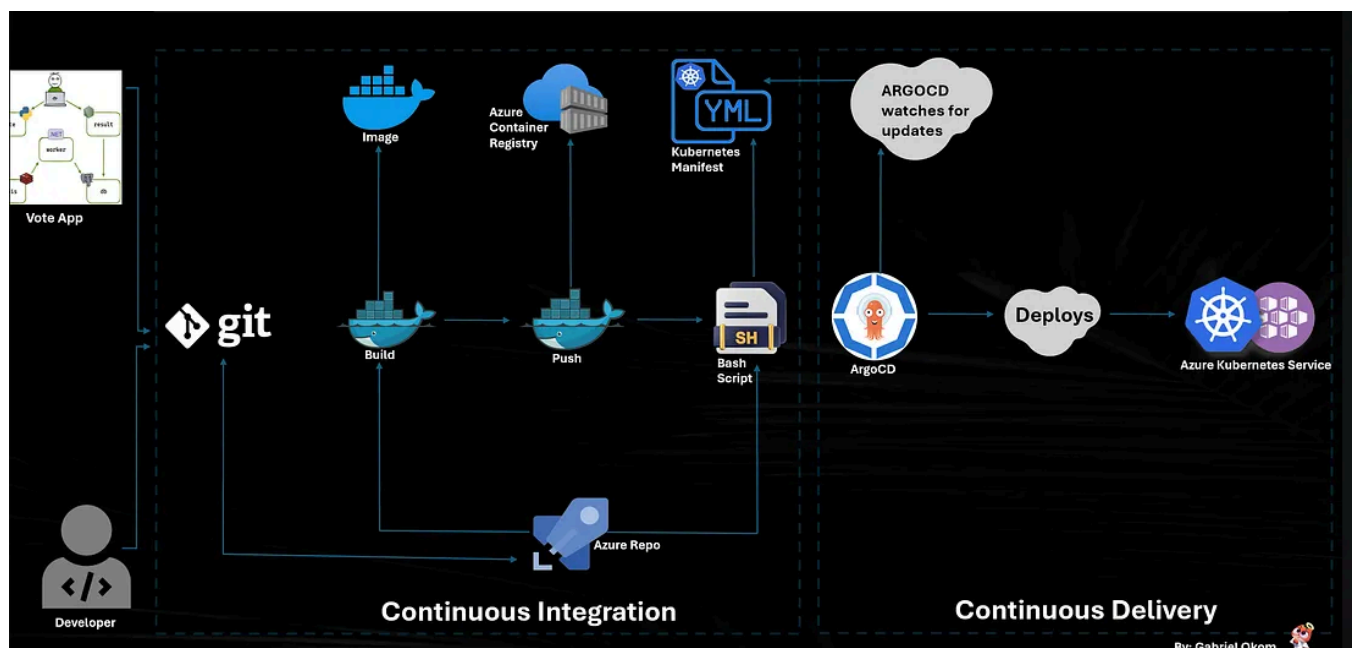


TABLE OF CONTENTS

Stage One: Continuous Integration

Step 1: Clone and Deploy the App Locally Using Docker-Compose

Step 2: Create an Azure DevOps Project and Import the Repo

Step 4: Set Up Self-Hosted Agent for the Pipeline

Step 5: Write a CI Pipeline Script for Each Microservice

Stage Two: Continuous Delivery

Step 1: Create an Azure Managed Kubernetes Cluster (AKS)

Step 2: Install AKS CLI and Set Up AKS for Use

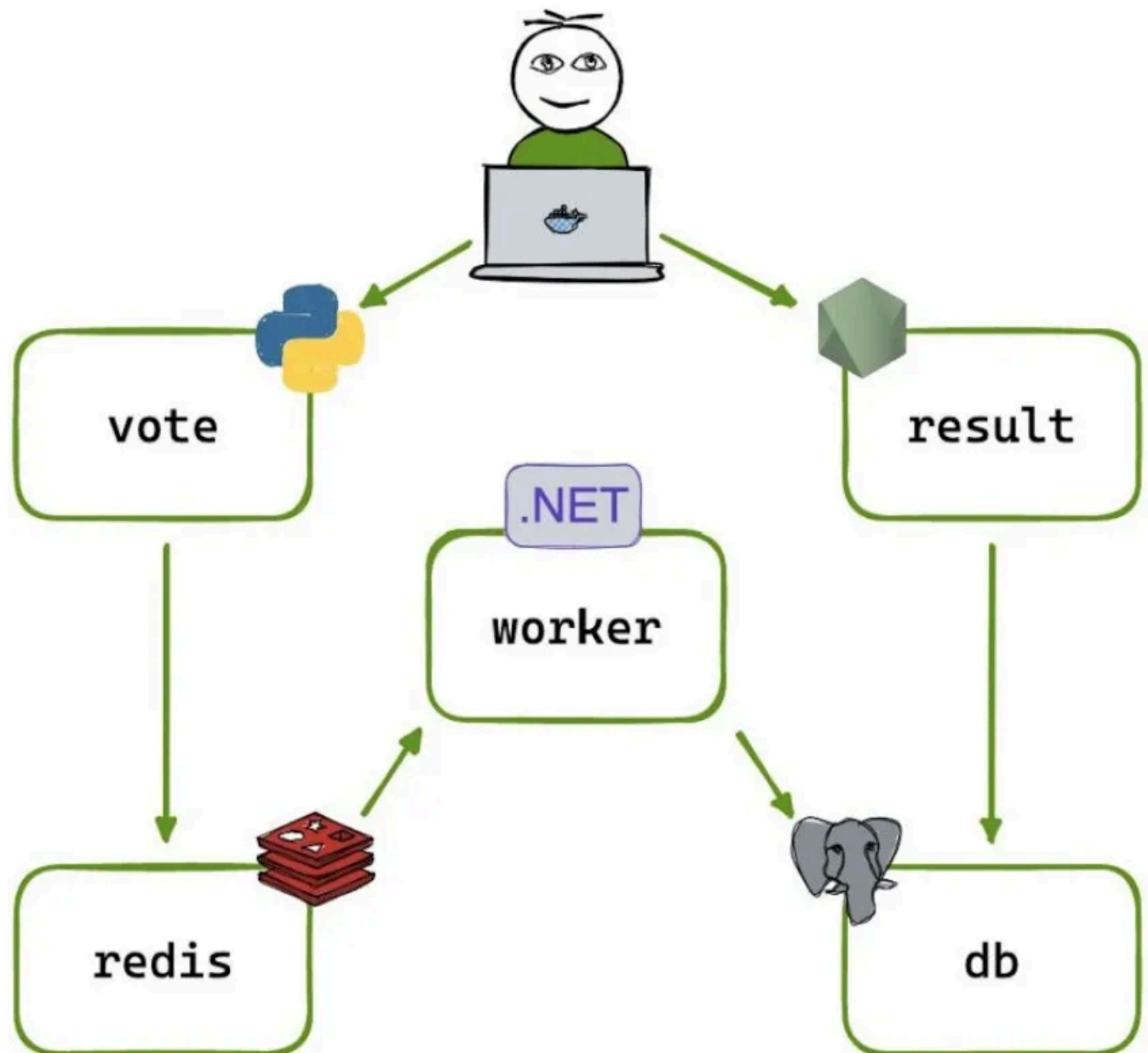
Step 3: Install ArgoCD

Step 4: Configure ArgoCD

Step 5: Write a Bash Script that updates the pipeline image on K8s manifest

Step 6: Create an ACR ImagePullSecret

Step 7: Verify the CICD process



The Docker Example Voting App is a microservices-based application, typically written in Python and Node.js, with the following components:

- 1> **Voting Frontend (Python/Flask):** Users cast votes via a simple web interface.
- 2> **Vote Processor Backend (Node.js/Express):** Receives and processes the votes.
- 3> **Redis Database (Redis):** Temporarily stores votes for fast access.
- 4> **Worker (Python/Flask):** Processes votes from Redis and updates the final count.

5> Results Frontend (Python/Flask): Displays real-time voting results.

PostgreSQL Database (PostgreSQL): Stores the final vote count.

How It Works:

The user votes on the frontend, the vote is processed and stored in Redis, then the worker updates the final count in PostgreSQL, and the results are shown on a results page. All components run in separate Docker containers for scalability and isolation.

Lets Begin...

1. Clone and deploy the App locally using docker-compose

To do this we have to create a VM to test our App locally,

a. Create an Azure Linux Ubuntu VM and Login:

Click [HERE](#) if you are not familiar with the steps needed in creating a Linux VM, When the VM is created, navigate to the folder and `ssh` into the machine using the command `ssh -i <your-key-name> azureuser@<your-ip-name>`

Note: Make sure `port 22` is open when creating the VM. (It usually opens by default for linux systems.)

b. Update the VM:

Run the command

```
sudo apt-get update
```

c. Install Docker and Docker-Compose

To be able to test the App locally on the VM, we will need to install docker onto our machine. This is needed for us to be able to run the `docker-compose up -d` command.

docker-compose is used to define and run multi container applications. such as microservices.

####1. Install required packages

```
sudo apt-get install \  
  ca-certificates \  
  curl \  
  gnupg \  
  lsb-release
```

####2. Add Docker's Official GPG Key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -c
```

####3. Set Up the Stable Repository

```
echo \  
  "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-  
    $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /
```

####4. Install Docker Engine

```
sudo apt-get update  
sudo apt-get install docker-ce docker-ce-cli containerd.io -y
```

####5. Install Docker Compose

```
sudo curl -L "https://github.com/docker/compose/releases/download/$(curl -s htt  
sudo chmod +x /usr/local/bin/docker-compose
```

####6. Manage Docker as a Non-Root User and refresh it

```
sudo usermod -aG docker $USER  
newgrp docker
```

####7. Check versions

```
sudo docker --version  
docker-compose --version
```

####8. Check you can docker command without sudo

```
docker ps
```

Note: docker ps output should be empty since no containers are running yet

d. Clone the Repo

Clone or fork the App [HERE](#) ; when this is done `cd` into the repo and run the `docker-compose up -d` command


```

azureuser@gabVoteApp:~$ git clone https://github.com/ougabriel/gab-example-voting-app.git
Cloning into 'gab-example-voting-app'...
remote: Enumerating objects: 1121, done.
remote: Total 1121 (delta 0), reused 0 (delta 0), pack-reused 1121 (from 1)
Receiving objects: 100% (1121/1121), 1.19 MiB | 16.40 MiB/s, done.
Resolving deltas: 100% (424/424), done.
azureuser@gabVoteApp:~$ ls
gab-example-voting-app
azureuser@gabVoteApp:~$ cd gab-example-voting-app/
azureuser@gabVoteApp:~/gab-example-voting-app$ ls
LICENSE      README.md      docker-compose.images.yml  docker-stack.yml  k8s-specifications  seed-data  worker
MAINTAINERS  architecture.excalidraw.png  docker-compose.yml         healthchecks      result              vote
azureuser@gabVoteApp:~/gab-example-voting-app$

```

Run the command to spin up all the containers at once.

```
docker-compose up -d
```

```

[+] Running 8/8
✔ Network gab-example-voting-app_back-tier Created 0.1s
✔ Network gab-example-voting-app_front-tier Created 0.1s
✔ Volume "gab-example-voting-app_db-data" Created 0.0s
✔ Container gab-example-voting-app-redis-1 Healthy 6.3s
✔ Container gab-example-voting-app-db-1 Healthy 6.3s
✔ Container gab-example-voting-app-result-1 Started 7.0s
✔ Container gab-example-voting-app-worker-1 Started 7.0s
✔ Container gab-example-voting-app-vote-1 Started 7.0s
azureuser@gabVoteApp:~/gab-example-voting-app$

```

This output shows our containers are all up and running.

```
docker ps
```

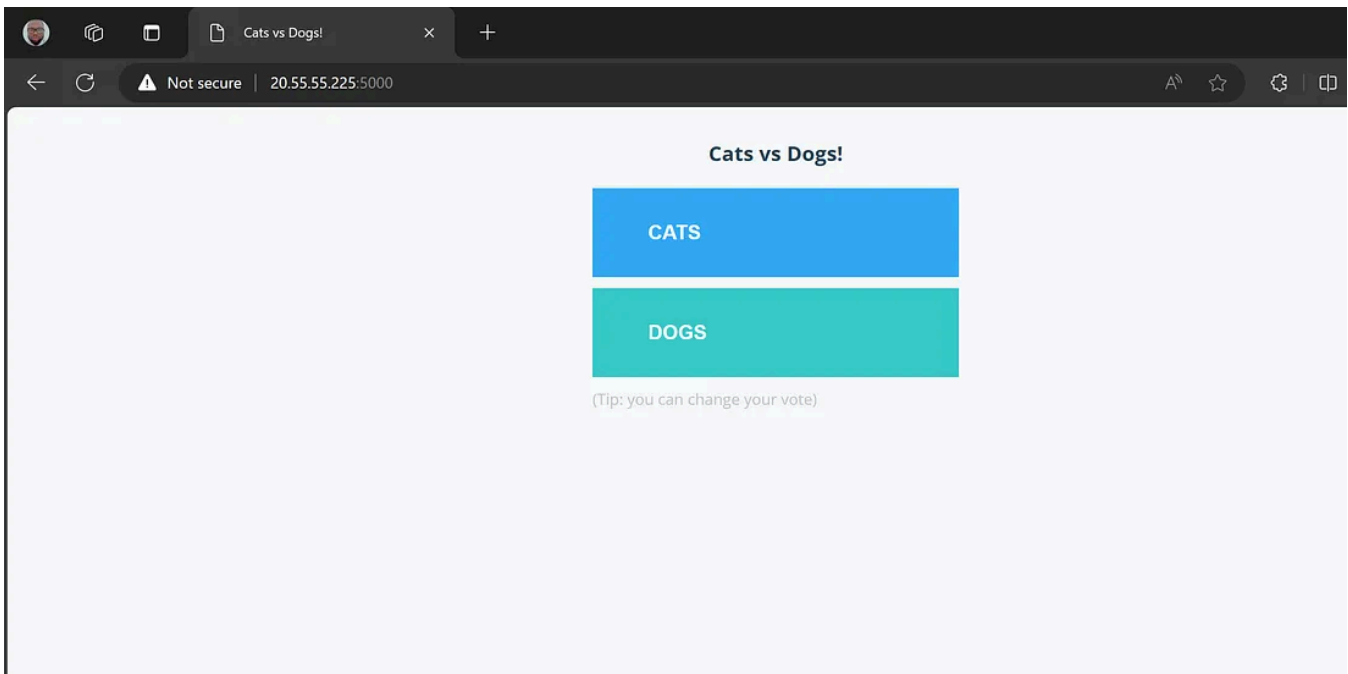
```

azureuser@gabVoteApp:~/gab-example-voting-app$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
6047b40df984   gab-example-voting-app-vote         "python app.py"         2 minutes ago Up About a minute (healthy) 0.0.0.0:5000->80/tcp, :::5000->80/tcp
b532a4960167   gab-example-voting-app-worker       "dotnet Worker.dll"     2 minutes ago Up About a minute          127.0.0.1:9229->9229/tcp, 0.0.0.0:5001->80/tcp, :::5001->80/tcp
6c78dbd48374   gab-example-voting-app-result       "nodemon --inspect=0..." 2 minutes ago Up 2 minutes (healthy) 5432/tcp
d83b9e52e102   postgres:15-alpine                 "docker-entrypoint.s..." 2 minutes ago Up 2 minutes (healthy) 6379/tcp
de19d5344a89   redis:alpine                        "docker-entrypoint.s..." 2 minutes ago Up 2 minutes (healthy)
azureuser@gabVoteApp:~/gab-example-voting-app$

```

From the output we can see the App container is mapped on port 5000 ; We can view the App in two (2) ways; to view the App do the following.

- Run `curl http://localhost:5000` to view the App on the terminal
- Copy the VM public address and type into the browser `http://vm-public-ip>:5000` to view the App.



Step 2: Create an Azure DevOps Project and Import the repo

a. > **Login:** Go to the url <https://dev.azure.com> and sign in to access the Azure DevOps page. *NOTE: If this is your first time, you may need to sign up and create an organization first.*

If this is your first time, click [HERE](#) to sign up for Azure DevOps pipeline

b. > **Create a Project:** I will be naming my project `votingApp` , when this is done. > Set the visibility to `private` , > Next, click on `Create Project`

Create a project to get started

Project name *
votingApp

Description
A voting App CICD deployment

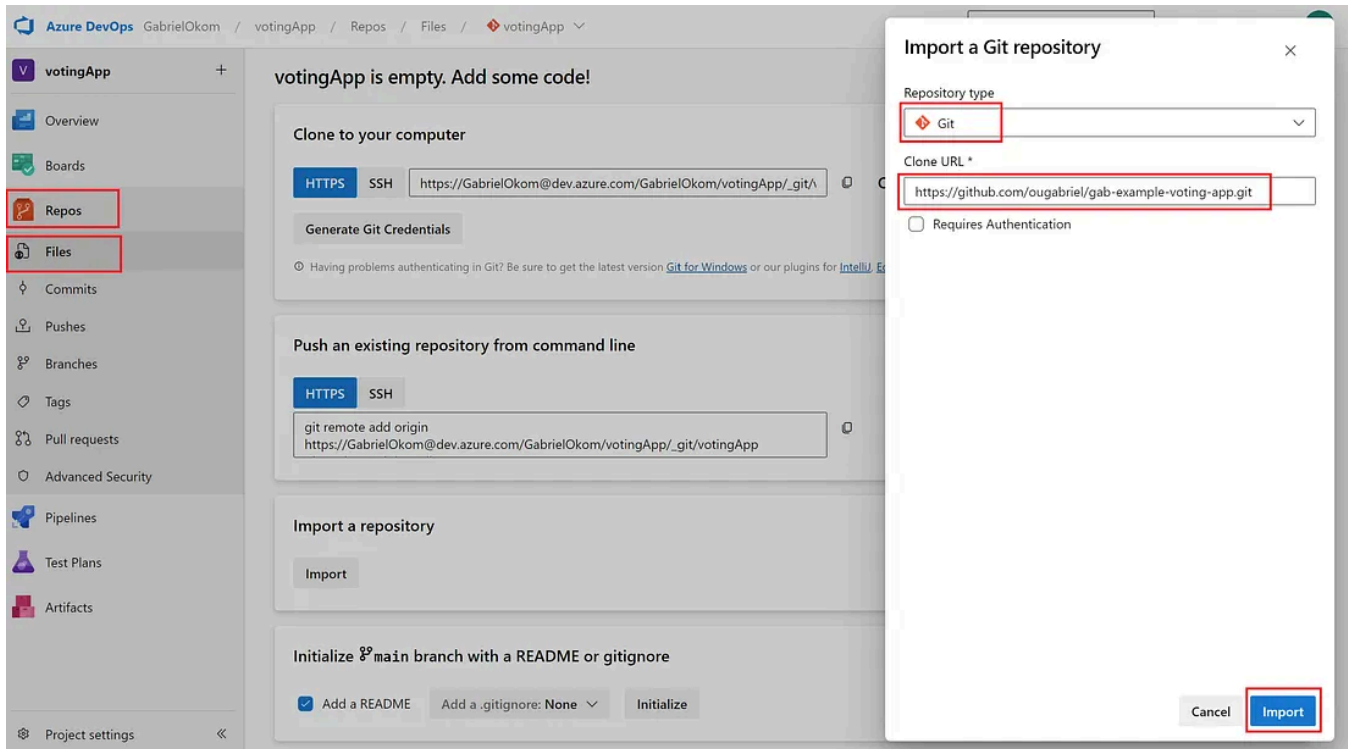
Visibility
☐ Public
Anyone on the internet can view the project. Certain features like TFVC are not supported.
☒ **Private**
Only people you give access to will be able to view this project.

Public projects are disabled for your organization. You can turn on public visibility with [organization policies](#).

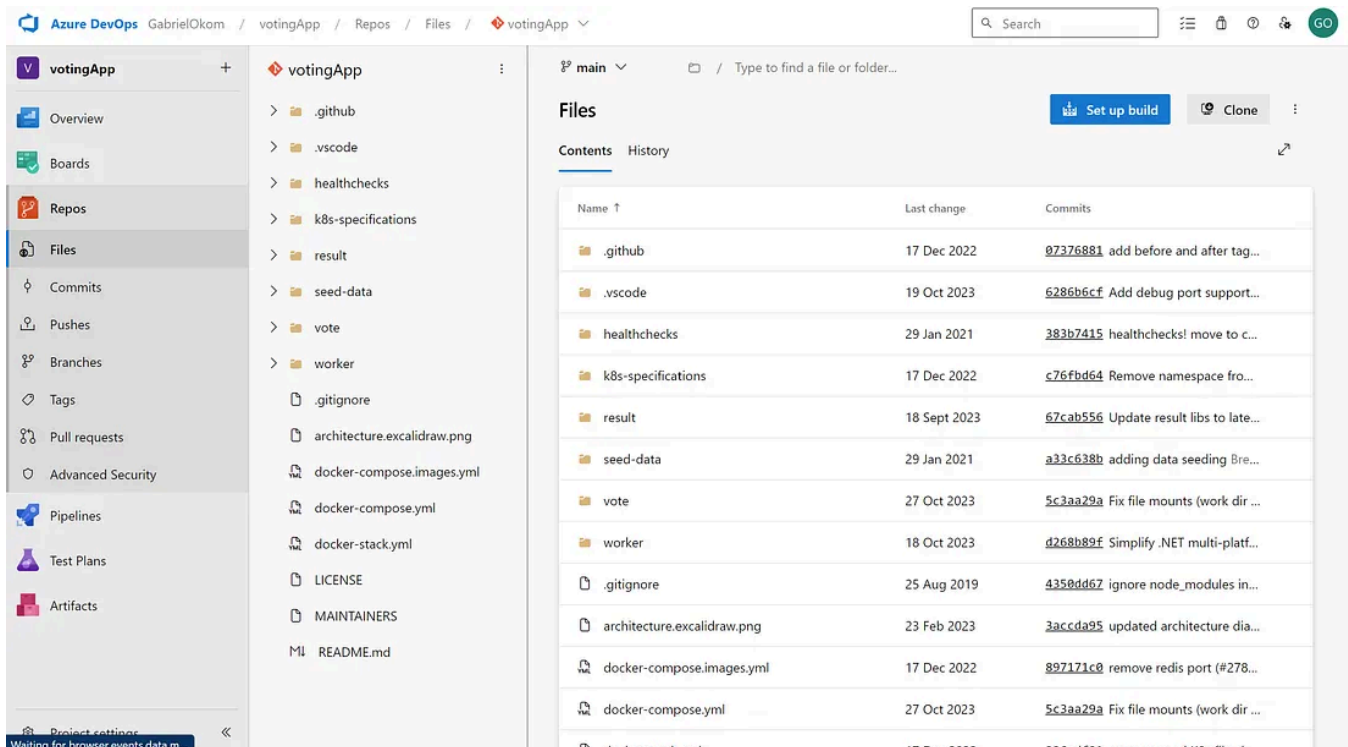
Advanced

+ Create project

c. > **Import the Git Repo to Azure Repo:** On the left menu pane, click on **Repos** > Then **Files** > In Repository type leave it as **Git** since that is where our code is stored. > Input the Git repo url > Click on **Import**



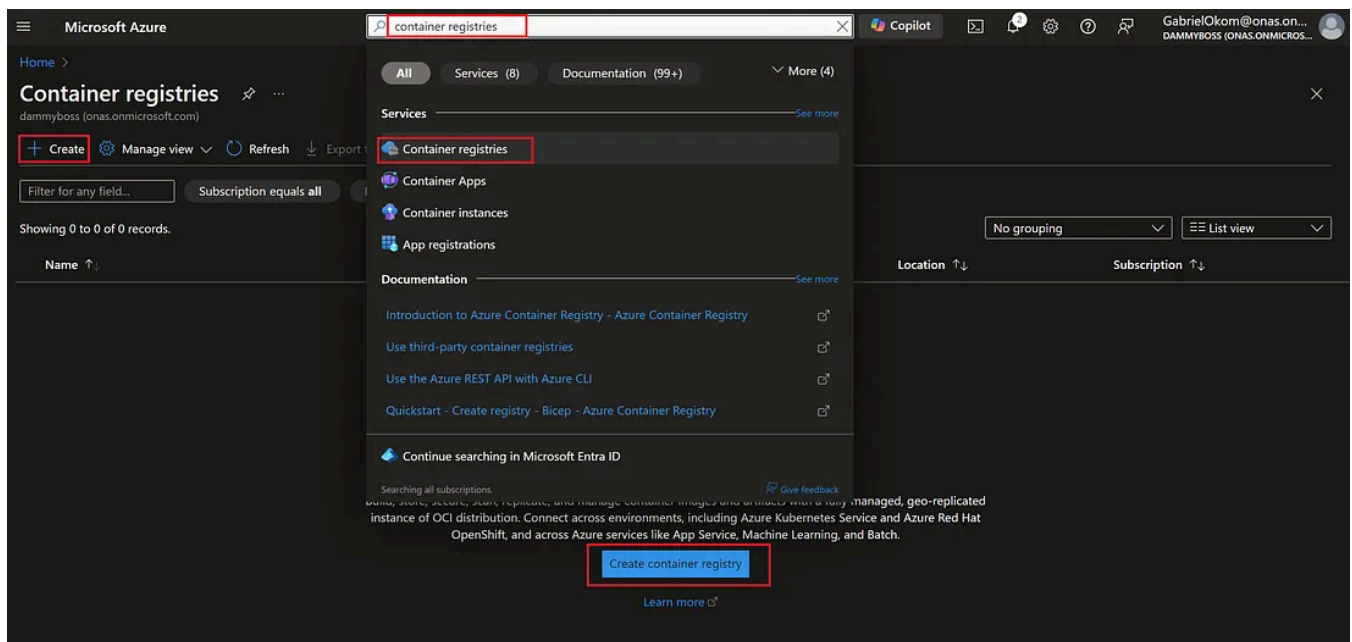
When this is done and successfully imported, we should see our entire Repo now.



Step 3. Create an Azure Container Registry

When we build our docker image, we need somewhere to push and store this image. We can also use DockerHub, but for the sake of this project we will be making use of the Azure Container Registry (ACR).

a. > Login to the azure portal > search for container registry click on it > on the left side, click on Create or Click on Create container registry at the bottom of the page.



b. > Choose a Resource group or create a new one if you dont have > Choose a name for your ACR > Select either the Basic or the Standard Pricing plan (optional) > Click on Review & create

Microsoft Azure

Home > Container registries > Create container registry

Basics Networking Encryption Tags Review + create

Azure Container Registry allows you to build, store, and manage container images and artifacts in a private registry for all types of container deployments. Use Azure container registries with your existing container development and deployment pipelines. Use Azure Container Registry Tasks to build container images in Azure on-demand, or automate builds triggered by source code updates, updates to a container's base image, or timers. [Learn more](#)

Project details

Subscription * Visual Studio Enterprise

Resource group * gabRG

[Create new](#)

Instance details

Registry name * gabVotingAppACR

Location * East US

Use availability zones ☐

Availability zones are activated on premium registries and in regions that support availability zones. [Learn more](#)

Pricing plan * Basic

[Review + create](#) < Previous Next: Networking >

After it is created, click on Go to resource to see the ACR page. > Take note of the ACR server name. We will be using it in step 5 when building and pushing the docker image . We need to choose the ACR name.

Microsoft Azure

Home > Microsoft.ContainerRegistry | Overview >

gabVotingAppACR Container registry

Search Move Delete

Overview

Activity log Access control (IAM) Tags Quick start Events Settings Services Repository permissions Policies Monitoring Automation Help

Essentials

Resource group (move) : gabRG

Location : East US

Subscription (move) : Visual Studio Enterprise

Subscription ID : 85c7908d-8028-4894-af5d-1991ee8f0450

Soft delete (Preview) : Disabled

Tags (edit) : Add tags

Login server : gabvotingappacr.azurecr.io

Creation date : 16/08/2024, 02:32 BST

Provisioning state : Succeeded

Pricing plan : Basic

Get started Monitoring Capabilities (9) Tutorials

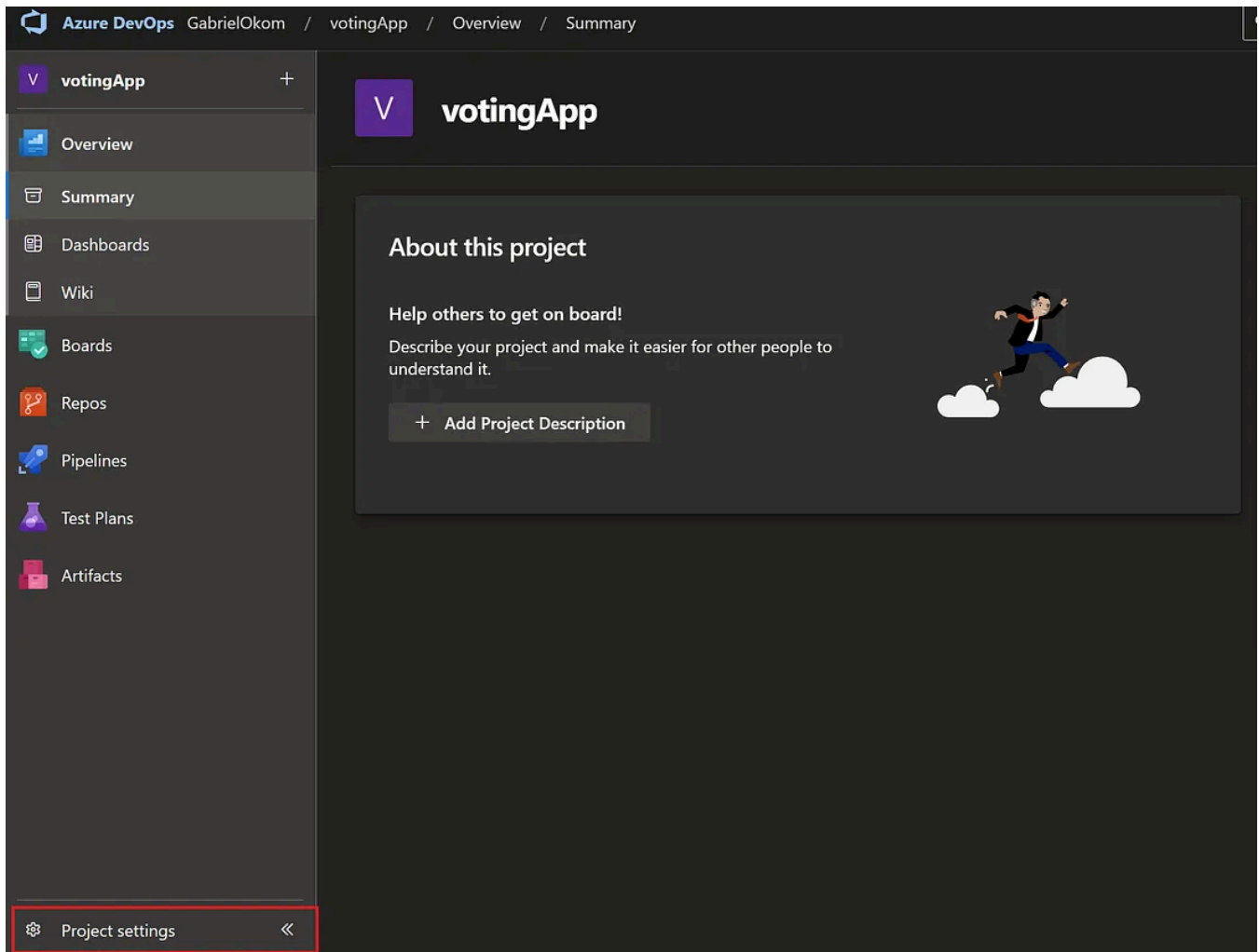
Simplify container lifecycle management

Container registry allows you to build, store, and manage container images and artifacts in a private registry for all types of container deployments. [Learn more](#)

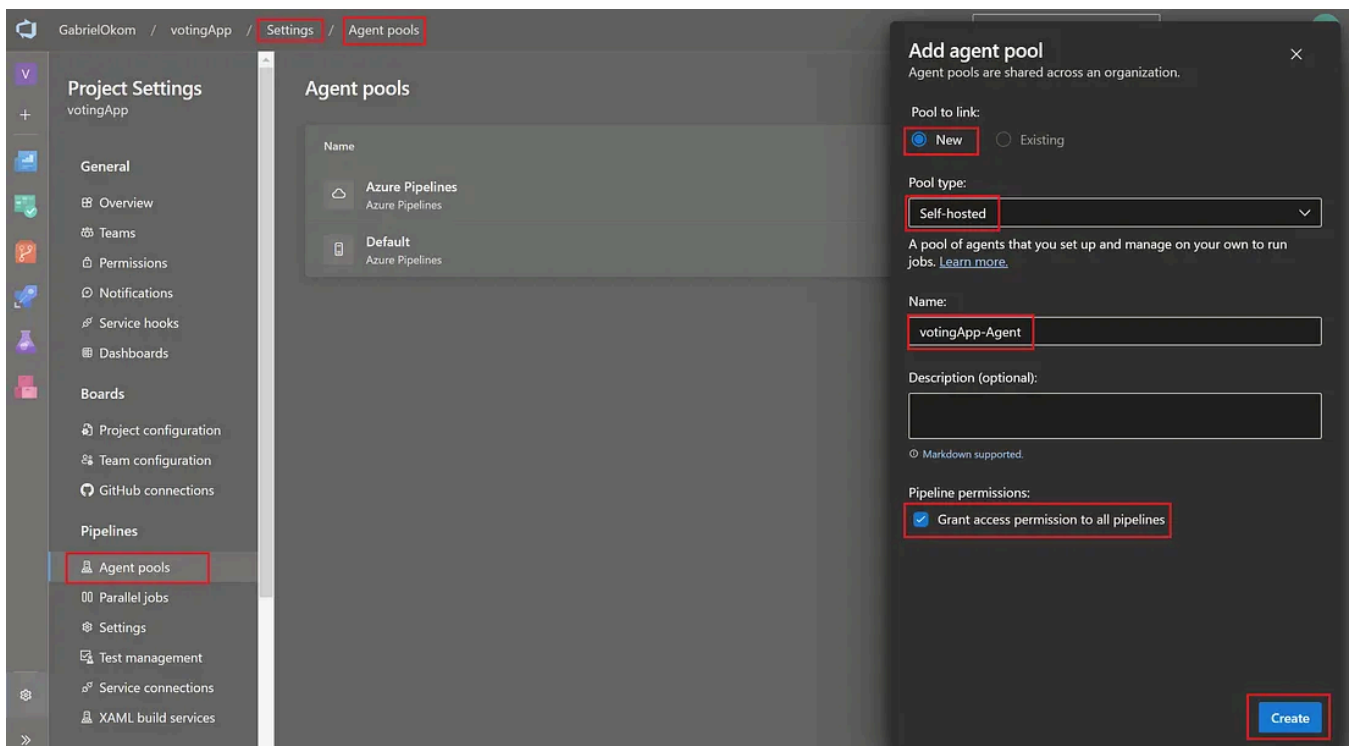
Step 4. Set up Self-Hosted Agent for the Pipeline

To save resources, we will be using the same VM used when deploying our App locally. (Note: you can create a new vm for this step, but make sure to install docker on it)

note: I will be going dark mode on my azure devops pipeline page from here.

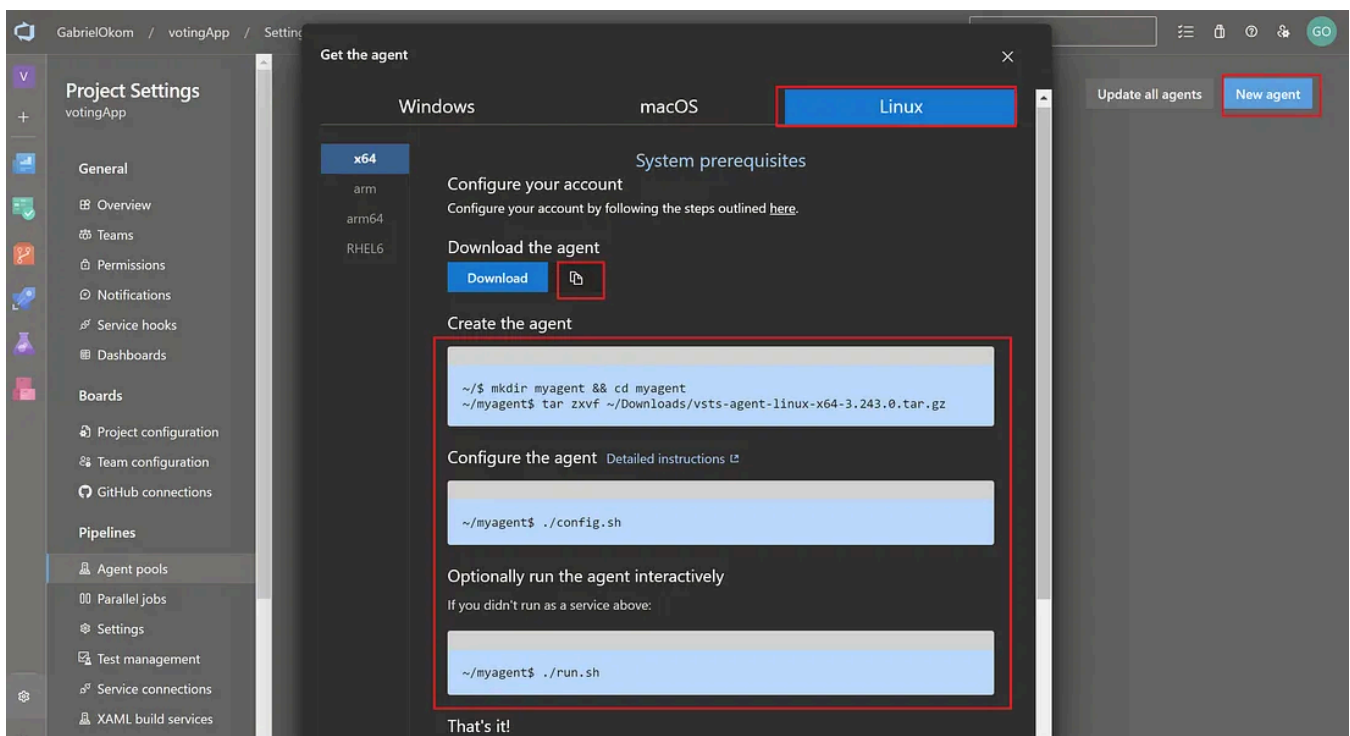


a. > to get started; go to your azure devops pipeline page > at the bottom left of the project page, click on `project settings` > on the left menu pane, click on `Agent pools` > On the right, Click on `Add pools` > for Pool to link, select `New` > Pool type should be `Self-hosted` > type in a name for your agent > for Pipeline permissions, check the `Grant access permissions to all pipelines` > then `Create`



b. > When the agent pool is created, click on it > on the right, click on the New agent button. > a new page pops up, in the page select the Linux system > copy and run the following codes shown in your VM.

Note: if you need a guide, agent script step by step command is given below



if you need more help, I have created a complete step by step comprehensive and well detailed guide on [how to set up a self-hosted agent in azure](https://ougabriel.medium.com/ci-cd-project-deploy-a-3-tier-microservice-voting-app-using-argocd-and-azure-devops-pipeline-1b3fb9d19138) >

<https://medium.com/@ougabriel/how-to-deploy-a-self-hosted-agent-in-azure-pipeline-e89250be09c8>

or take a queue from this steps to build your own self-hosted agent.

```
#1. create a dir and move into the dir
mkdir myagent && cd myagent

#2. download the agent, (please copy and paste the url from the "Download the a
sudo apt install wget -y
wget https://vstsagentpackage.azureedge.net/agent/3.243.0/vsts-agent-linux-x64-

#3. extract the download the agent files
tar zxvf vsts-agent-linux-x64-3.243.0.tar.gz

#4 configure the agent
./config.sh

#5. start the agent and keep it running
./run.sh
```

c. > when you run the `./config.sh` script > for the URL prompt, copy and paste your azure devops url link; example `https://dev.azure.com/<org name>` > hit enter to choose PAT > create and paste your PAT (Personal Access Token) > type in the name of the agent pool created earlier > for the next prompts hit enter for defaults > run `./run.sh` to start the agent and keep it listening for jobs.


```

azureuser@gabVoteApp:~/myagent$ ./config.sh

Azure Pipelines
agent v3.243.0 (commit d429c24)

>> End User License Agreements:

Building sources from a TFVC repository requires accepting the Team Explorer Everywhere End User License Agreement. This step is not required for building sources from Git repositories.

A copy of the Team Explorer Everywhere license agreement can be found at:
/home/azureuser/myagent/license.html

Enter (Y/N) Accept the Team Explorer Everywhere license agreement now? (press enter for N) > Y

>> Connect:

Enter server URL > https://dev.azure.com/GabrielOkom
Enter authentication type (press enter for PAT) > ydswdj32bc36kageeruvnwnafuh3gedmjon73mhqui4f665wobqa
Enter a valid value for authentication type.
Enter authentication type (press enter for PAT) >
Enter personal access token > *****
Connecting to server ...

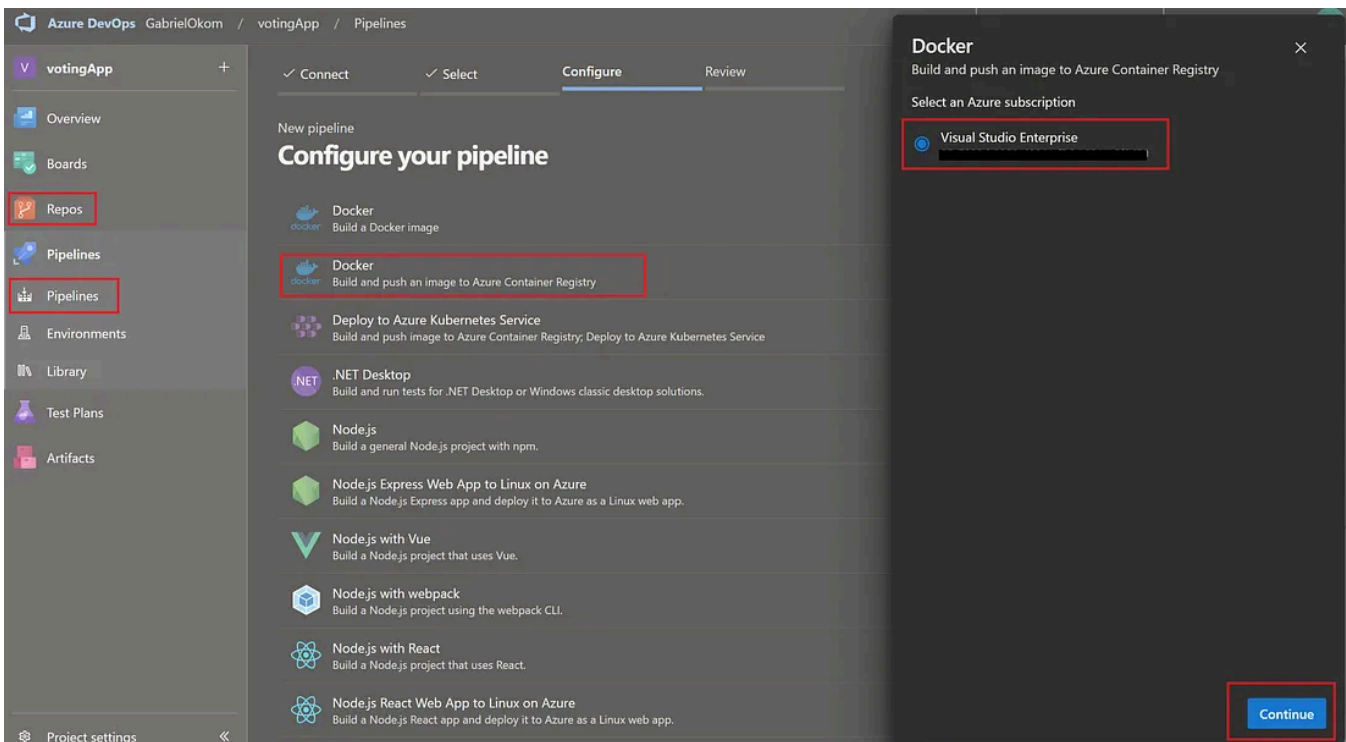
>> Register Agent:

Enter agent pool (press enter for default) > votingApp-Agent
Enter agent name (press enter for gabVoteApp) >
Scanning for tool capabilities.
Connecting to the server.
Successfully added the agent
Testing agent connection.
Enter work folder (press enter for _work) >
2024-08-16 02:51:57Z: Settings Saved.
azureuser@gabVoteApp:~/myagent$ ./run.sh
Scanning for tool capabilities.
Connecting to the server.
2024-08-16 02:54:38Z: Listening for Jobs

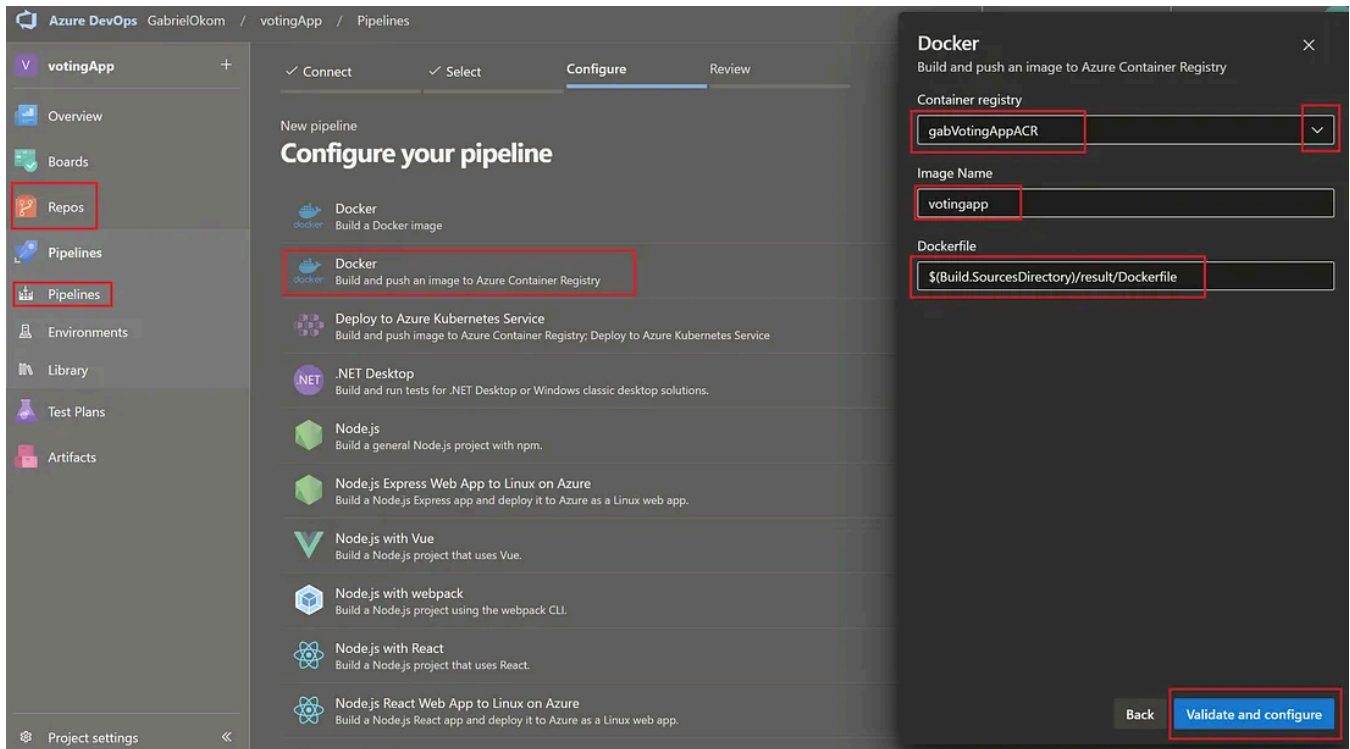
```

Step 5. Write a CI pipeline Script for each of the Microservices using separate build and push stages

a. > Setting up the pipeline: To do this, on the left menu panel click on **Repo** > then click on **pipeline** > in the pop window select the docker that helps us to push the image to Azure Container Registry > Choose the Azure subscription you will be using



After clicking the 'Continue' button, a new pop windows appear as shown below. > from the drop down menu select the name of your azure container registry created earlier > select or input the image name > since we are buiding a docker image, this page will be automatically populated but make sure it is using the result repo file because we are building the result image first, when this is done we can then continue with the other services.



b.> Create the pipeline scripts: We will start with the `vote` microservice. the script will be divided into two stages (1. Build and 2. Push). Make sure your pipeline script looks exactly like this

```
# Docker
# Build and push an image to Azure Container Registry
# https://docs.microsoft.com/azure/devops/pipelines/languages/docker

trigger:
  paths:
    include:
      - vote/*      #the repo we are using

resources:
  - repo: self

variables:
  # Container registry service connection established during pipeline creation
  dockerRegistryServiceConnection: '37868c72-32ef-488d-a490-1415f4b73792'
```

```

imageRepository: 'votingapp'
containerRegistry: 'gabvotingappacr.azurecr.io'
dockerfilePath: '$(Build.SourcesDirectory)/vote/Dockerfile'
tag: '$(Build.BuildId)'

# Agent VM image name
pool:
  name: 'votingApp-Agent'    #the name of my newly created selfhosted agent

stages:
- stage: Build
  displayName: Build the Voting App
  jobs:
  - job: Build
    displayName: Build

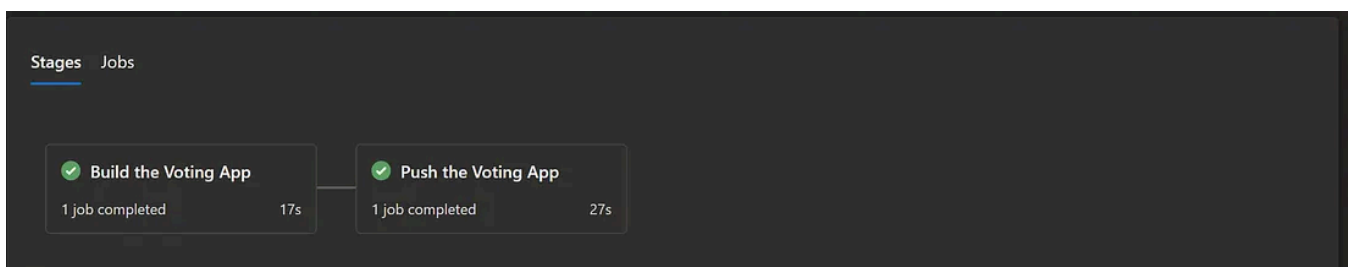
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'gabVotingAppACR'
        repository: 'votingapp/vote'
        command: 'build'
        Dockerfile: 'vote/Dockerfile'

- stage: Push
  displayName: Push the Voting App
  jobs:
  - job: Push
    displayName: Pushing the voting App
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'gabVotingAppACR'
        repository: 'votingapp/vote'
        command: 'push'

```

Note: Make sure to change the *agent VM name* to match the name of your self-hosted agent.

c. When this is done run the pipeline.



Next: We will run the pipeline script for the other microservices which are the `result` and `worker` services.

Now, to build and push the other two microservices (`result` and `worker`).

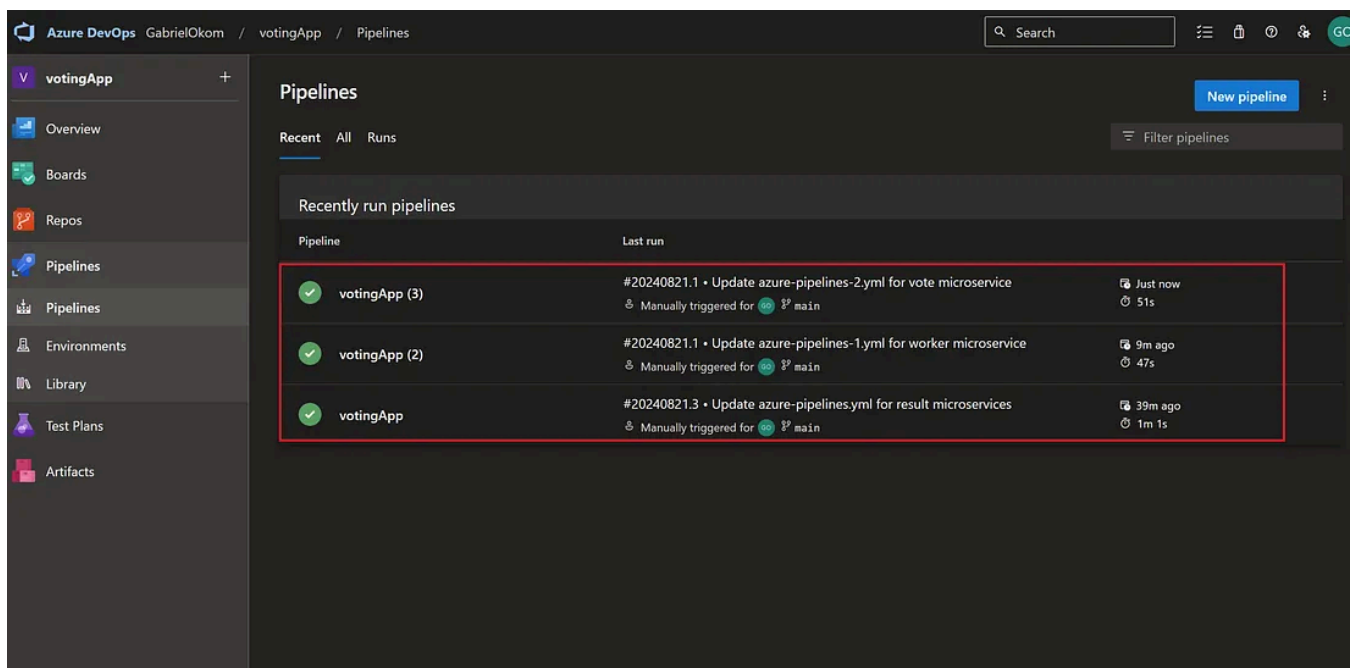
Repeat **STEP 5** but this time you are changing anywhere you see the service name `vote` to `worker` and `result` in the pipeline scripts. Change to `result` build and push; then change to `worker` build and push the image.

(if you are not sure what to do here i have added the complete pipeline scripts at the end of this project)

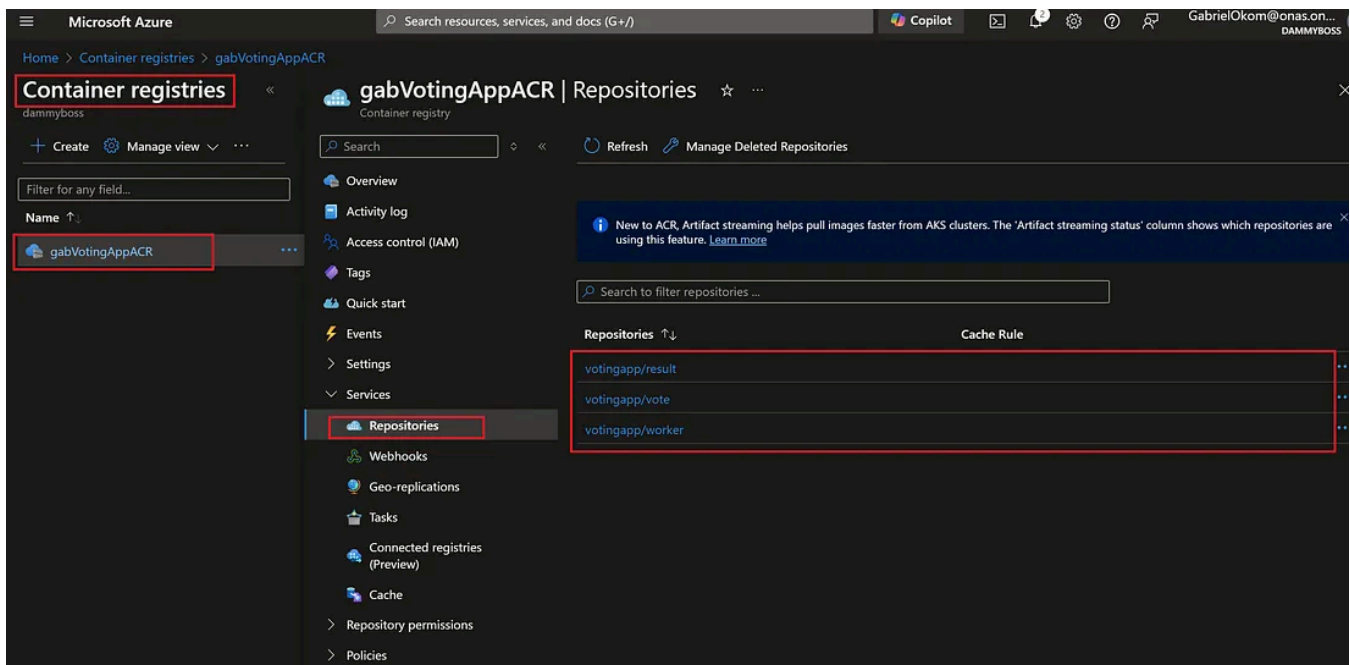
At the end of the pipeline run scripts, you should have three (3) images pushed into your `azure container registry` as shown below.

d. Confirm the push:

The pipeline...



The Azure Container Registry... go to azure portal > click on container registry > click on the name of the `ACR` you created for this project > click on `repositories` to view all the images pushed into it.



Stage Two: Continuous Delivery

Step 1: Create an Azure Managed Kubernetes Cluster (AKS)

In the [Azure Portal](#) search for AKS or Azure Kubernetes Service and click **Create** > In Project details, Select your subscription and the resource group where you'd like to deploy the kubernetes cluster into > For cluster details, leave it as **Dev/Test** > Choose a name for your kubernetes cluster > select a region of your choice (if you face a resource deployment issues in **us-east** change it > for Availability Zones, choose **Zones 1** > and leave every other settings as default.

Click **Next** > In Node Pools, check the **agentpool** and click on it > In the **agentpool** pop window, change the 'scale method' from manual to automatic > change the **min** and **max** node count to 1 and 2 > Click on **Enable public IP per node** > click **update**

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

Home > Azure Kubernetes Service (AKS) > Create Kubernetes cluster >

Update node pool ...

votingApp-k8s

Node pool name *

agentpool

Mode *

User

System

The primary node pool must be a system node pool to support system pods.

OS SKU *

Azure Linux

Ubuntu Linux

Windows 2022

Windows 2019

Linux is required for system node pools.

Availability zones

Zone 1

Enable Azure Spot instances

Azure Spot instances cannot be used with system node pools.

Node size *

Standard D4ds v5

4 vcpus, 16 GiB memory

Choose a size

Scale method

Manual

Autoscale - Recommended

This option is recommended so that the cluster is automatically sized correctly for the current running workloads.

Minimum node count *

1

Maximum node count *

2

The maximum node count allowed for an AKS cluster is 1000 per node pool and 5000 nodes across all node pools in this cluster.

Optional settings

Max pods per node *

110

30 - 250

Enable public IP per node

☒

Labels

Labels are key/value pairs that can be used to categorize or add identifying information to Kubernetes resources such as nodes. Labels for the node pool will be applied to each node in the node pool. [Learn more](#)

Key

Value

Update

Cancel

> Leave every thing else as default and click on Review & create

This will take sometime to fully deploy

Home > microsoft.aks-1724290202600 | Overview >

votingApp-k8s

Kubernetes service

Search

Create

Connect

Start

Stop

Delete

Refresh

Open in mobile

Give feedback

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Microsoft Defender for Cloud

Cost analysis

Kubernetes resources

Settings

Monitoring

Automation

Help

Essentials

Resource group

gabRG

Status

Succeeded (Running)

Subscription

Visual Studio Enterprise

Location

West US 2

Subscription ID

85c7908d-8028-4894-af5d-1991ee8f0450

Tags

Add tags

Kubernetes version

1.29.7

API server address

votingapp-k8s-dns-scvmp1jt.hcp.westus2.azmk8s.io

Network configuration

Azure CNI Overlay

Node pools

1 node pool

Container registries

Attach a registry

Get started

Properties

Monitoring

Capabilities (5)

Recommendations (0)

Tutorials

Kubernetes services

Encryption type

Encryption at-rest with a platform-managed key

Virtual node pools

Not enabled

Node pools

Node pools

1 node pool

Kubernetes versions

1.29.7

Node sizes

Standard_D4ds_v5

Configuration

Kubernetes version

1.29.7

Networking

API server address

votingapp-k8s-dns-scvmp1jt.hcp.westus2.azmk8s.io

Network configuration

Azure CNI Overlay

Pod CIDR

10.244.0.0/16

Service CIDR

10.0.0.0/16

DNS service IP

10.0.0.10

Docker bridge CIDR

-

Cilium dataplane

Not enabled

Network Policy

None

Load balancer

Standard

HTTP application routing

Not enabled

Step 2: Install Azure CLI and Set Up AKS for use

We will be using the following commands to quickly install the AKS CLI and set up our newly deployed AKS cluster for use through a newly created VM. This VM will serve as a workstation for us to connect to our AKS and ArgoCD

Note: > Create a new Azure VM and run the following script inside of it > Make sure to change Resource_Group and AKS_name to the name of your Resource Group and AKS cluster respectively.

```
# Update the package list and install necessary dependencies
sudo apt-get update
```

```
# Step 1: Install Azure CLI
echo "Installing Azure CLI..."
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

```
# Step 2: Log in to Azure
az login --use-device-code
```

Since we are using the command `az login --use-device-code` a prompt will appear with a url and login authentication code . Copy the url into a browser and paste the code into the login page to finish installing the `az cli`

```
azureuser@votingAppVM2:~$ az login --use-device-code
To sign in, use a web browser to open the page https://microsoft.com/devicelogin and enter the code EAX96KDQ8 to authenticate.
Retrieving tenants and subscriptions for the selection...
[Tenant and subscription selection]
No      Subscription name      Subscription ID      Tenant
```

Verify: > To be sure this correctly done. type `az --version` to verify the installation

```
# Step 3: Install kubectl
sudo az aks install-cli
```



```
# Step 4: Get AKS credentials
# Replace <ResourceGroupName> and <AKSClusterName> with your actual resource gr
RESOURCE_GROUP="<ResourceGroupName>"      #change here only
AKS_NAME="<AKSClusterName>"               #change here only

az aks get-credentials --resource-group gabRG --name votingApp-k8s --overwrite-
```

```
# Step 5: Verify the connection
kubectl get nodes
```

```
azureuser@votingAppVM2:~$ az aks get-credentials --resource-group gabRG --name votingApp-k8s --overwrite-existing
Merged "votingApp-k8s" as current context in /home/azureuser/.kube/config
azureuser@votingAppVM2:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-agentpool-90021581-vmss000001  Ready    <none>   7m37s  v1.29.7
azureuser@votingAppVM2:~$
```

In the image above, we have a single node cluster with a `ready` STATUS.

Step 3: Install ArgoCD

We will install `argocd` using a bash script to make it quicker, script details has been explained below to help you get better understanding

```
#!/bin/bash

# Step 1: Install Argo CD
echo "Installing Argo CD..."

# Create a namespace for Argo CD
kubectl create namespace argocd

# Install Argo CD in the argocd namespace
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/s

# Step 2: Wait for all Argo CD components to be up and running
echo "Waiting for Argo CD components to be ready..."
kubectl wait --for=condition=Ready pods --all -n argocd --timeout=600s

# Step 3: Get the initial admin password
echo "Retrieving the Argo CD initial admin password..."
ARGOCD_INITIAL_PASSWORD=$(kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d)
echo "Argo CD initial admin password: $ARGOCD_INITIAL_PASSWORD"
```

```
# Step 4: Access the Argo CD server
echo "Exposing Argo CD server via NodePort..."
kubectl -n argocd patch svc argocd-server -p '{"spec": {"type": "NodePort"}}'

# Retrieve the Argo CD server URL
ARGOCD_SERVER=$(kubectl -n argocd get svc argocd-server -o jsonpath='{.spec.clu
ARGOCD_PORT=$(kubectl -n argocd get svc argocd-server -o jsonpath='{.spec.ports
echo "You can access the Argo CD server at http://$ARGOCD_SERVER:$ARGOCD_PORT"

# Step 5: Login to Argo CD CLI (Optional)
echo "Installing Argo CD CLI..."
sudo curl -sSL -o /usr/local/bin/argocd https://github.com/argoproj/argo-cd/releases
sudo chmod +x /usr/local/bin/argocd

echo "Logging into Argo CD CLI..."
argocd login $ARGOCD_SERVER:$ARGOCD_PORT --username admin --password $ARGOCD_IN

echo "Argo CD installation and setup complete!"
```

Script Overview:

- **Step 1:** Installs Argo CD in a namespace called `argocd`.
- **Step 2:** Waits for all Argo CD components to be ready.
- **Step 3:** Retrieves the initial admin password for Argo CD.
- **Step 4:** Exposes the Argo CD server using a NodePort service type, making it accessible via a URL.
- **Step 5:** Optionally installs the Argo CD CLI and logs in using the initial admin password.

How to Use:

1. Save this script to a file, for example, `install-argo-cd.sh`.
2. Make the script executable: `chmod +x install-argo-cd.sh`.
3. Run the script: `./install-argo-cd.sh`.

This will set up Argo CD on your Kubernetes cluster and provide you with the necessary details to access and manage it.

Get GABRIEL OKOM's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

a. Set up Port Rule for ArgoCD

Since `argocd` is already exposed to the internet via its `NodePort`. We can access it on the browser by using the IP address of our VM and the `port` address of the `http` port 80 service which is 31436 as shown below for my own cluster.

```
azureuser@votingAppVM2:~$ kubectl get svc -n argocd
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
argocd-applicationset-controller    ClusterIP   10.0.34.60    <none>         7000/TCP,8080/TCP  109s
argocd-dex-server                   ClusterIP   10.0.172.250  <none>         5556/TCP,5557/TCP,5558/TCP  109s
argocd-metrics                      ClusterIP   10.0.114.141  <none>         8082/TCP          109s
argocd-notifications-controller-metrics ClusterIP   10.0.171.119  <none>         9001/TCP          109s
argocd-redis                        ClusterIP   10.0.150.146  <none>         6379/TCP          109s
argocd-repo-server                  ClusterIP   10.0.221.118  <none>         8081/TCP,8084/TCP  109s
argocd-server                       NodePort    10.0.218.239  <none>         80 31436/TCP,443:31272/TCP  109s
argocd-server-metrics               ClusterIP   10.0.102.218  <none>         8083/TCP          109s

azureuser@votingAppVM2:~$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE              KERNEL-VERSION   CONTAINER-RUNTIME
aks-agentpool-90021581-vmss000001 Ready     <none>    16m   v1.29.7   10.224.0.4    52.250.58.123 Ubuntu 22.04.4 LTS    5.15.0-1070-azure     containerd://1.7.20-1
```

Before we do this, we will need to expose the port 31436 of our kubernetes cluster > Go to Azure Portal and search for VMSS (virtual machine scale set) > click on network settings > Click Creat Port Rule > In the drop down, click on Inbound Rule > Add the Port number

The screenshot shows the Azure portal interface for a Virtual Machine Scale Set (VMSS) named 'aks-agentpool-90021581-vmss'. The 'Network settings' tab is selected, and the 'Add inbound security rule' dialog is open. The dialog is configured with the following settings:

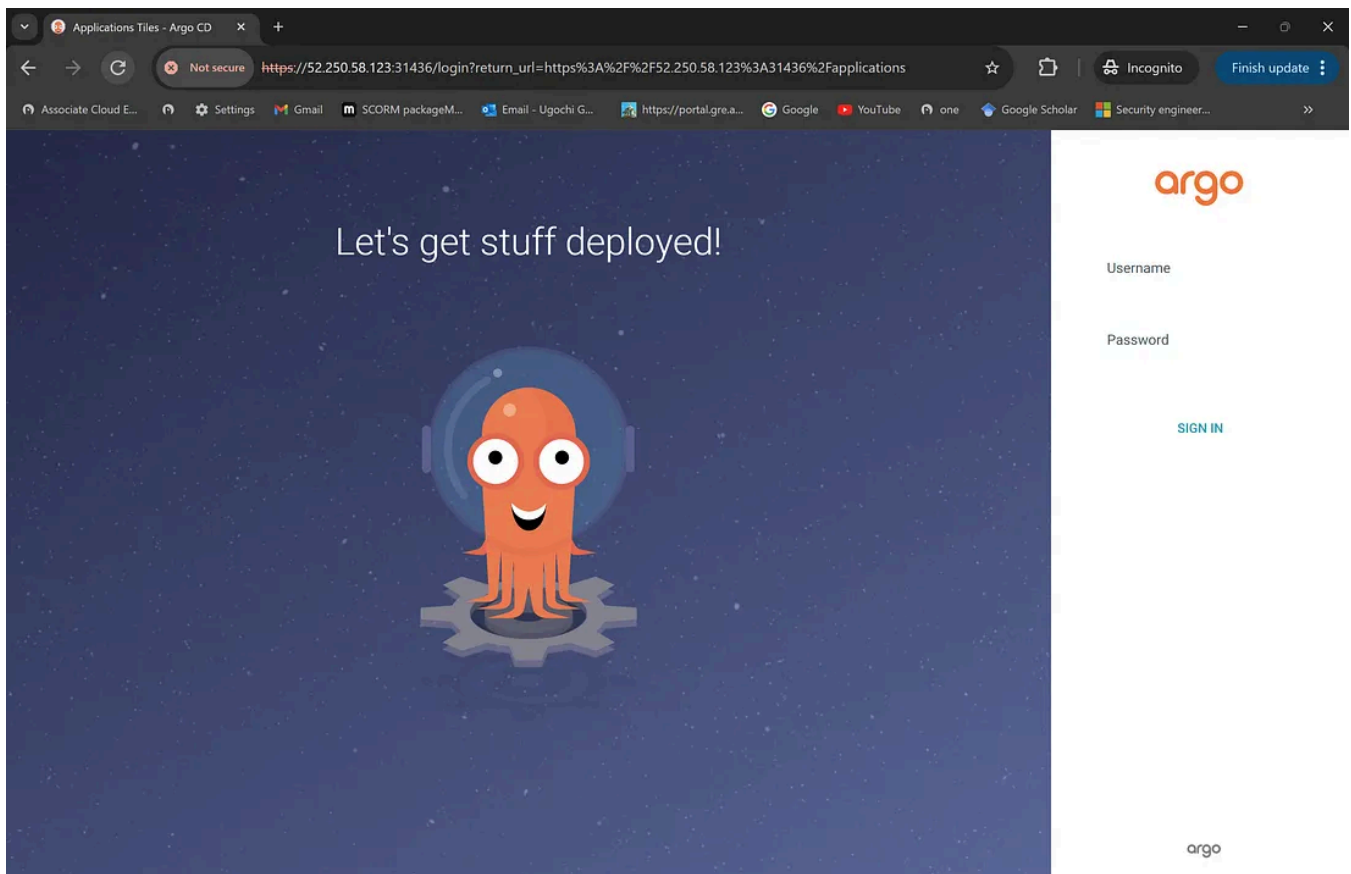
- Source:** Any
- Source port ranges:** *
- Destination:** Any
- Service:** Custom
- Destination port ranges:** 31436
- Protocol:** Any
- Action:** Allow
- Priority:** 100
- Name:** (empty)

The 'Add' button at the bottom right of the dialog is highlighted with a red box.

type in the ip address and port number into a browser. Example:

`http://52.250.58.123:31436`

You should get the argoCD signin landing page



b.> Login to ArgoCD: We need to run the following command to get the login credentials of argoCD.

> Check to see the argoCD secret pod `kubectl get secret -n argocd`

```
azureuser@votingAppVM2:~$ kubectl get secret -n argocd
NAME                                TYPE      DATA      AGE
argocd-initial-admin-secret         Opaque    1          35m
argocd-notifications-secret        Opaque    0          35m
argocd-redis                        Opaque    1          35m
argocd-secret                       Opaque    5          35m
```

> Edit the secret `kubectl edit argocd-initial-admin-secret -n argocd`

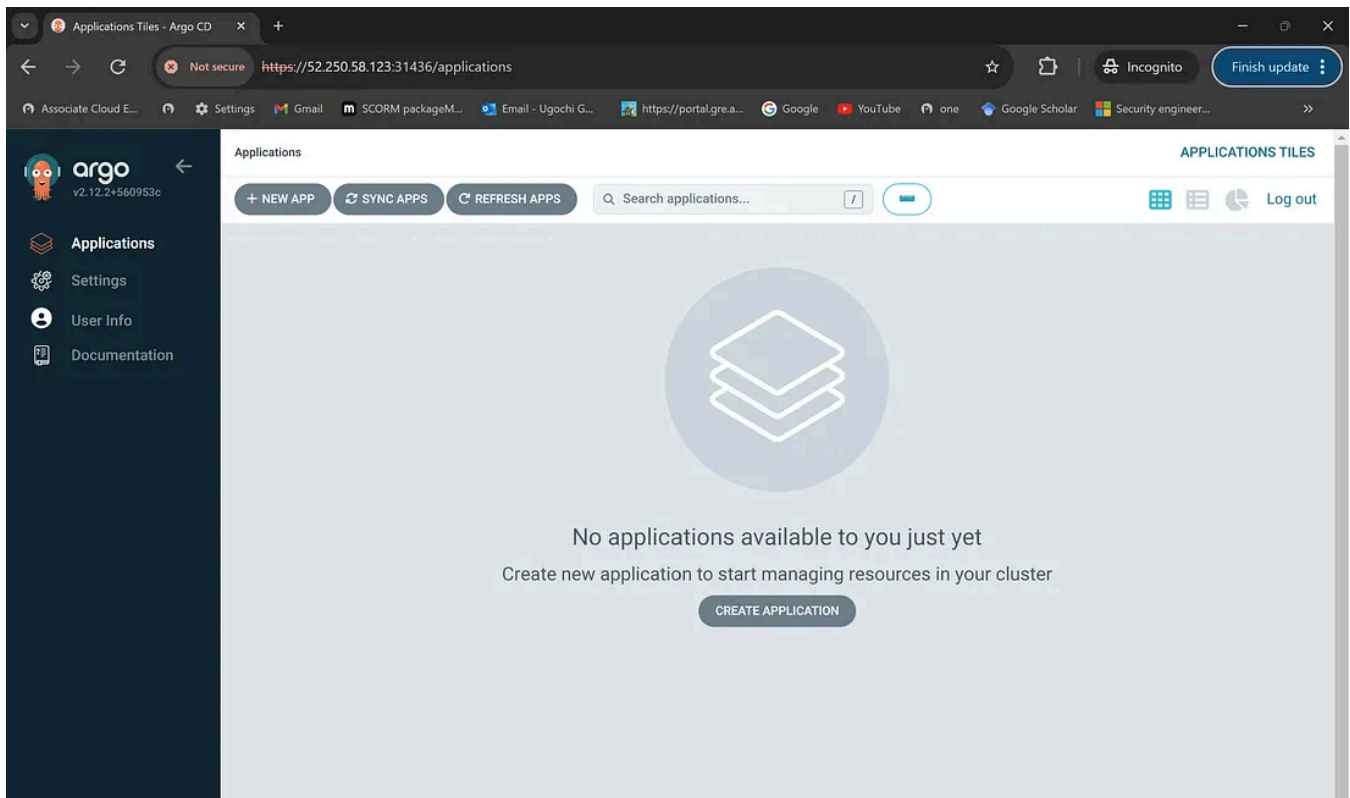
```
## Please edit the object below. Lines beginning with a '#' will be ignored,  
# and an empty file will abort the edit. If an error occurs while saving this file will be  
# reopened with the relevant failures.  
#  
apiVersion: v1  
data:  
  password: LUNqbmRSZ2dCZmVTY1h1Ug==  
kind: Secret  
metadata:  
  creationTimestamp: "2024-08-26T21:21:02Z"  
  name: argocd-initial-admin-secret  
  namespace: argocd  
  resourceVersion: "7534"  
  uid: 820d6c74-e605-4218-9248-4d8e1df03965  
type: Opaque  
~  
~  
~  
~  
~  
~  
~
```

> Copy the password and run the following command on the terminal; because the password is base64 encoded we will need to decode it

```
echo <the argoCD password> | base64 --decode
```

Login to the argoCD using the following

```
username: admin  
password: <your decoded argocd password>
```

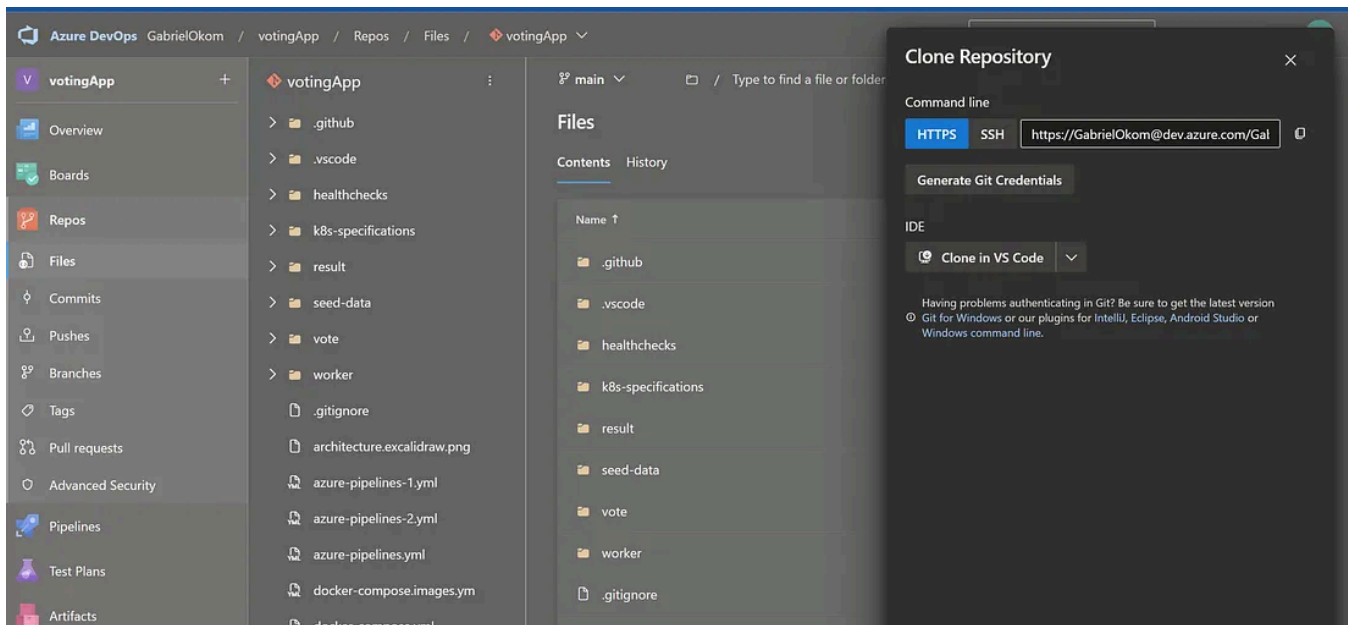


Step 4: Configure ArgoCD

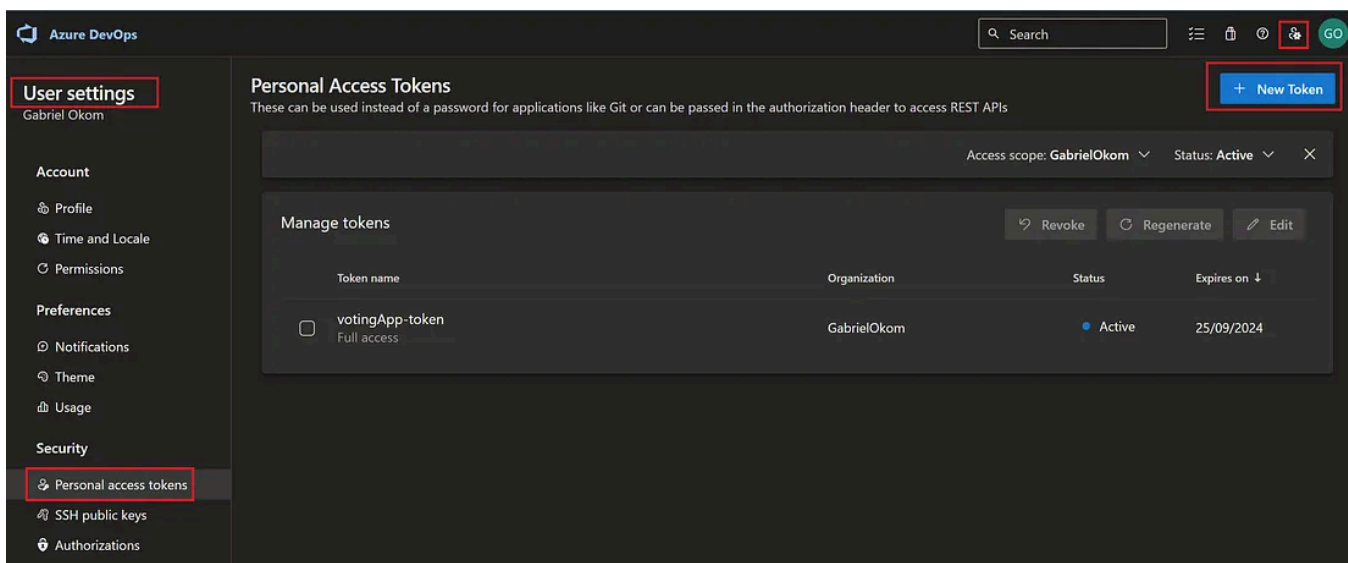
We need to configure argoCD to point to our azure repo where our `kubernetes` manifest files are stored. ArgoCD will help us install these manifest files in our kubernetes cluster and also help to monitor for changes in the Azure Repo. Any changes made or observed in the repo will then be pushed to our kubernetes cluster.

To do this, we need to connect argoCD to Azure repo and to our AKS.

a. > Connect to Azure repo: copy the HTTPS URL of your Azure repo and paste it in the URL field.



> Next, create a Personal Access Token > In your azure repo, click on user settings > from the drop down menu, click on personal access token > click New Token > Give it a name and a permission, it can be read access or full access *Note: full access is for testing purpose only* > Copy the newly created PAT



> Connect the ArgoCD to azure repo: In your argoCD page, click on settings > click on the +Connect repo button > in choose your connection method select VIA HTTPS > for project, select default > paste the URL of the Azure Repo > Edit the azure repo and paste your PAT. > test if connection is successful by clicking on CONNECT

Edit the Azure Repo to also contain the Personal Access Token, for example

BEFORE

https://GabrielOkom@dev.azure.com/GabrielOkom/votingApp/_git/votingApp

AFTER

https://<INSERT-PERSONAL-ACCESS-TOKEN-HERE>@dev.azure.com/GabrielOkom/votingApp/_git/votingApp

CONNECT

SAVE AS CREDENTIALS TEMPLATE

CANCEL

Choose your connection method:

VIA HTTPS

CONNECT REPO USING HTTPS

Type

git

Project

default

Repository URL

https://sb5bkwnfxhrxjldlh2mtc6inauo5qdcjufqqjwzt5n5z46sdq@dev.azure.com/GabrielOkom/votingApp/_git/votingApp

Username (optional)

Password (optional)

check connection

Settings / Repositories

REPOSITORIES

+ CONNECT REPO

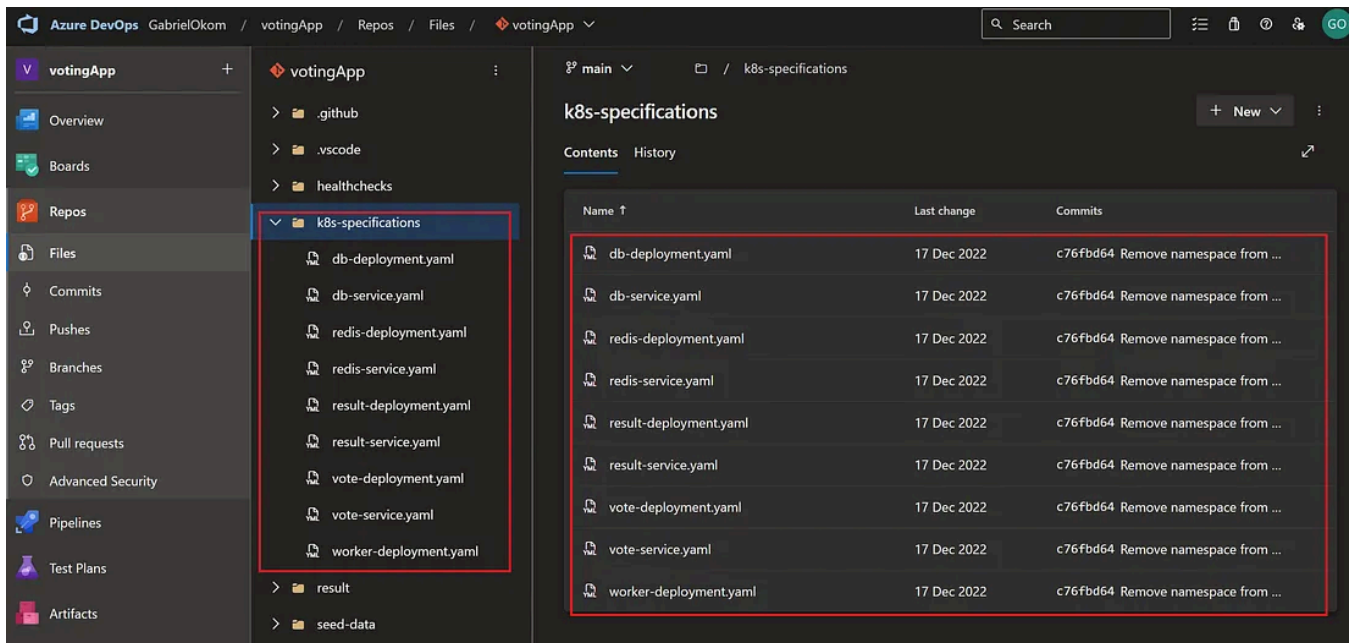
REFRESH LIST

Log out

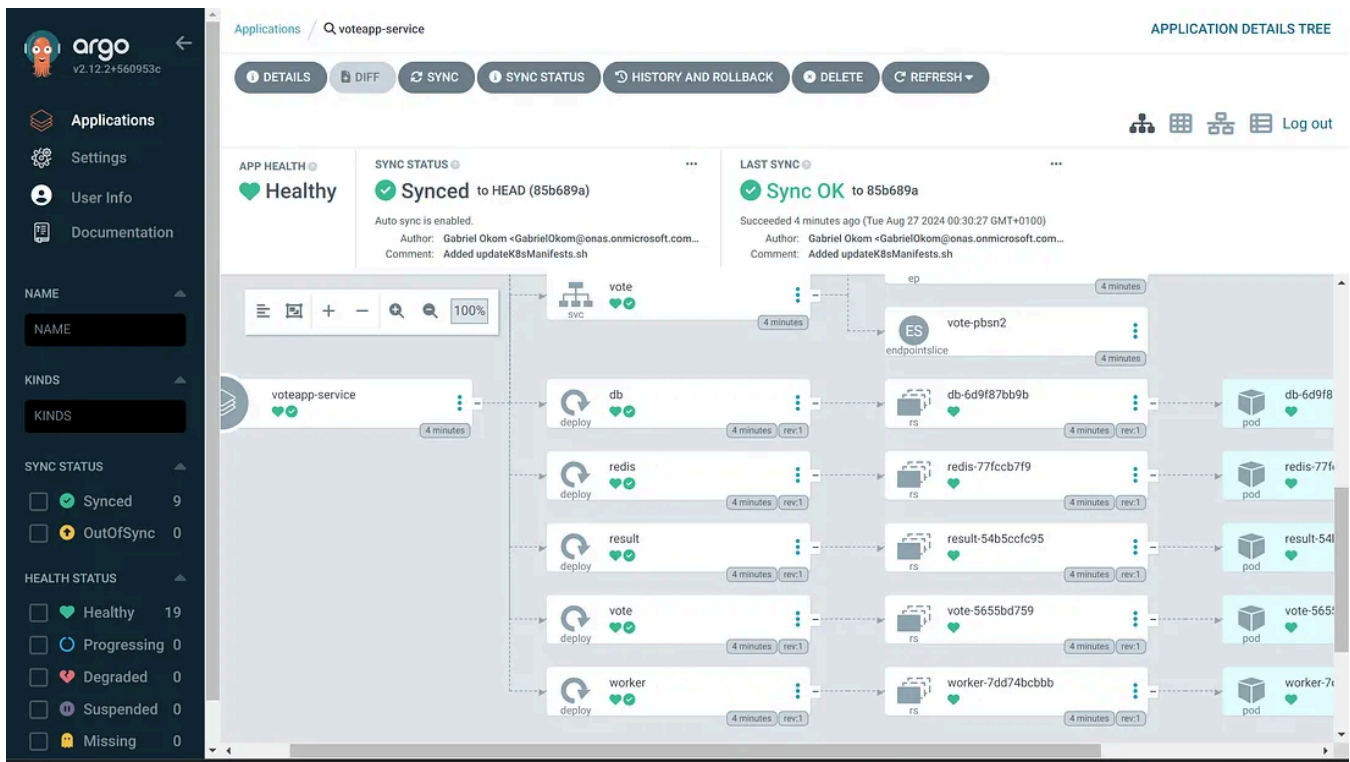
	TYPE	NAME	PROJECT	REPOSITORY	CONNECTION STATUS
	git		default	https://jyrbwbn3kiw2zighhszubv6w5mvqdzqi6yohh...	Successful

b. > Connect to Azure Kubernetes Service (AKS) > In the argoCD page, click on **New Application** > for **Application Name**, give it a name > in **project**, select **default** > for **SYNC POLICY** choose **automatic** > for **Repository URL** source, click on the Azure Repo URL added earlier, *it will appear automatically* > for **Path**, type in **k8s-specifications** > in **namespace**, type **default** > create it and wait for all the manifest files to be deployed to k8s and the pods running

k8s-specifications is our Azure repo folder where all our kubernetes manifest files are located, We want argoCD to deploy and monitor this folder for us



From argoCD we can confirm if all the pods are running



We can also check through our terminal to confirm the running pods.

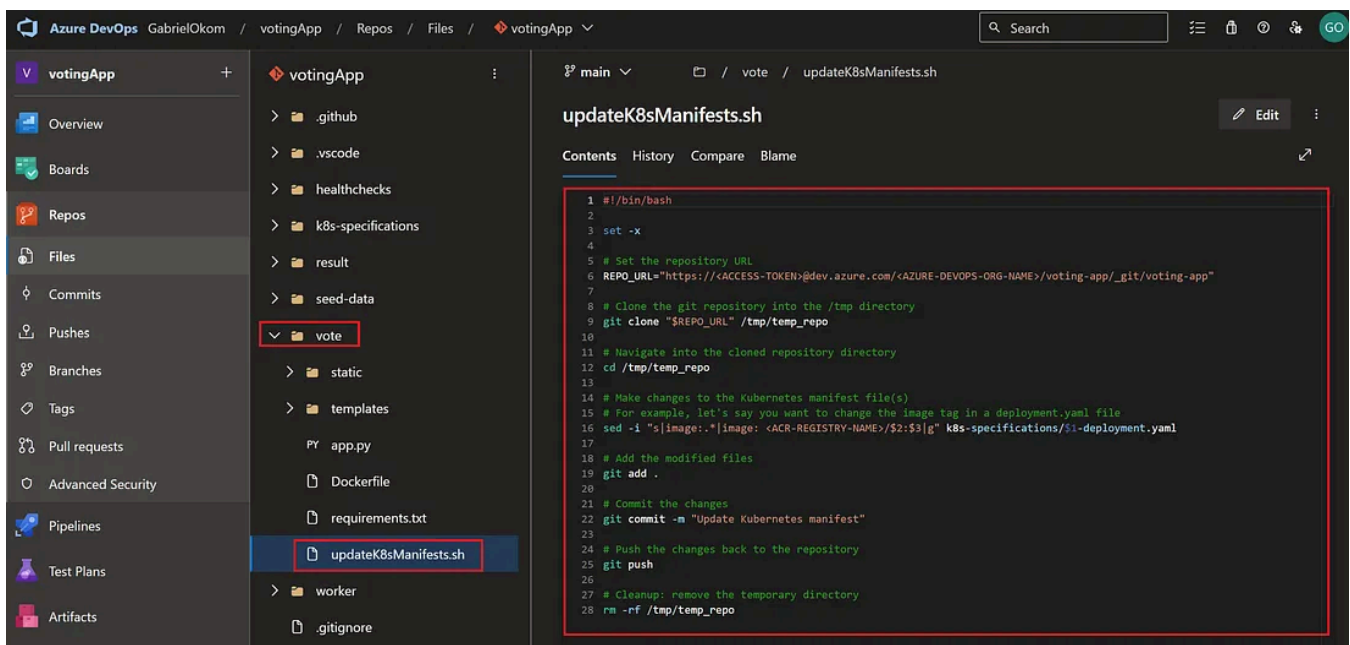


Step 5: Write a Script that updates the pipeline image on Kubernetes manifest

This a crucial step to our CICD project, in the CD (Continuous Delivery) stage we have the argoCD monitoring the `k8s-manifest` files in the Azure Repo and deploying the changes to AKS cluster; but *how does argoCD know when there is a change in the CI stage?* . (Remember, in the CI stage we built the image and pushed the image to ACR (Azure Container Registry)).

Now, in order to automate the CI stage with the CD stage, we will be making use of BASH Script. This `bash` script will monitor the ACR for any changes, when there is a new CI build and push it will detect the change and help us update this change to the Azure Repo (*Remember, argoCD is already monitoring the Azure repo*)

a.> Add the script to the `vote-service` folder within the repo



the script

```
#!/bin/bash

set -x

# Set the repository URL
REPO_URL="https://<ACCESS-TOKEN>@dev.azure.com/GabrielOkom/votingApp/_git/votir

# Clone the git repository into the /tmp directory
git clone "$REPO_URL" /tmp/temp_repo

# Navigate into the cloned repository directory
cd /tmp/temp_repo
```

```
# Make changes to the Kubernetes manifest file(s)
# For example, let's say you want to change the image tag in a deployment.yaml
sed -i "s|image:.*|image: <ACR NAME>/$2:$3|g" k8s-specifications/$1-deployment.

# Add the modified files
git add .

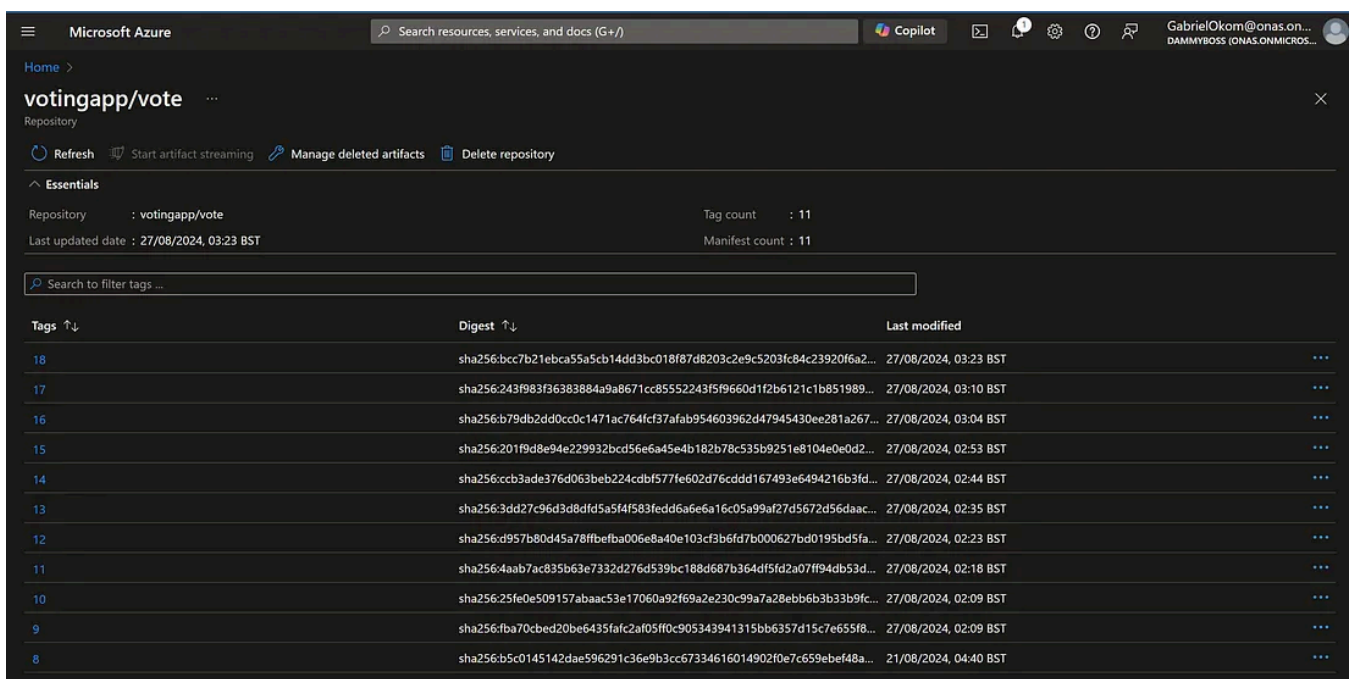
# Commit the changes
git commit -m "Update Kubernetes manifest"

# Push the changes back to the repository
git push

# Cleanup: remove the temporary directory
rm -rf /tmp/temp_repo
```

This Bash script automates cloning a Git repository, updating a Kubernetes deployment's Docker image tag, committing the changes, and pushing them back to the repository; it takes three parameters: `\$1` (deployment file prefix which is `vote`), `\$2` (image name), and `\$3` (new image tag).

what will happen here is the script will detect the new image inside the container registry using its image tag. *(every new build generates a new image tag for that build)*



Tags ↑↓	Digest ↑↓	Last modified
18	sha256:bcc7b21ebca55a5cb14dd3bc018f87d8203c2e9c5203fc84c23920f6a2...	27/08/2024, 03:23 BST
17	sha256:243f983f36383884a9a8671cc85552243f5f9660d1f2b6121c1b851989...	27/08/2024, 03:10 BST
16	sha256:b79db2dd0cc0c1471ac764fcf37afab954603962d47945430ee281a267...	27/08/2024, 03:04 BST
15	sha256:201f9d8e94e229932bcd56e6a45e4b182b78c535b9251e8104e0e0d2...	27/08/2024, 02:53 BST
14	sha256:ccb3ade376d063beb224cbbf577fe602d76cdd167493e6494216b3fd...	27/08/2024, 02:44 BST
13	sha256:3dd27c96d3d8dfd5a5f4f583fedd6a6e6a16c05a99af27d5672d56daac...	27/08/2024, 02:35 BST
12	sha256:d957b80d45a78fbefba006e8a40e103cf3b6fd7b000627bd0195bd5fa...	27/08/2024, 02:23 BST
11	sha256:4aab7ac835b63e7332d276d539bc188d687b364df5fd2a07f94db53d...	27/08/2024, 02:18 BST
10	sha256:25fe0e509157abaac53e17060a92f69a2e230c99a7a28ebb6b3b33b9fc...	27/08/2024, 02:09 BST
9	sha256:fba70cbcd20be6435fafc2af05f0c905343941315bb6357d15c7e655f8...	27/08/2024, 02:09 BST
8	sha256:b5c0145142dae596291c36e9b3cc67334616014902f0e7c659ebef48a...	21/08/2024, 04:40 BST

when this is done it updates the k8s-manifest deployment file with the new image and new tag > argocd detects the changes and deploys it to AKS > this way we can have a new version of the app whenever a change is made

it changes the image section of the `vote-deployment.yaml` file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: vote
  name: vote
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vote
  template:
    metadata:
      labels:
        app: vote
    spec:
      containers:
        - image: gabvotingappacr.azurecr.io/votingapp/vote:18 ##tag changes anyti
          name: vote
          ports:
            - containerPort: 80
              name: vote
```

b.> Add a new stage to the `vote-service` pipeline

```
# Docker
# Build and push an image to Azure Container Registry
# https://docs.microsoft.com/azure/devops/pipelines/languages/docker

trigger:
  paths:
    include:
      - vote/*

resources:
  - repo: self

variables:
  # Container registry service connection established during pipeline creation
  dockerRegistryServiceConnection: 'a777b3f1-28d4-40f3-bbdf-0904d5c89545'
  imageRepository: 'votingapp'
  containerRegistry: 'gabvotingappacr.azurecr.io'
  dockerfilePath: '$(Build.SourcesDirectory)/result/Dockerfile'
```



```

    tag: '$(Build.BuildId)'

pool:
  name: 'votingApp-Agent'

stages:
- stage: Build
  displayName: Build the voting App
  jobs:
  - job: Build
    displayName: Build

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'gabVotingAppACR'
        repository: 'votingapp/vote'
        command: 'build'
        Dockerfile: '**/Dockerfile'

- stage: Push
  displayName: Push the voting App
  jobs:
  - job: Push
    displayName: Push

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: 'gabVotingAppACR'
        repository: 'votingapp/vote'
        command: 'push'

#####
##### Add the script below to the vote-service pipeline

- stage: Update_bash_script
  displayName: update_Bash_script
  jobs:
  - job: Updating_repo_with_bash
    displayName: updating_repo_using_bash_script
    steps:
    - task: ShellScript@2
      inputs:
        scriptPath: 'vote/updateK8sManifests.sh'
        args: 'vote $(imageRepository) $(tag)'

```

`\$1` (vote), `\$2` (image name), and `\$3` (new image tag).

Now that we have finally integrated the `shell` script to the CI and now have a completely setup CICD pipeline, we can now go ahead and test our deployment

NOTE: if the update script runs and not yet updated we can edit the configmap of the argocd to make fetch and deploy updates quicker, run `kubectl edit cm argocd-cm -n argocd` and add `data: timeout-reconciliation` to be 10s

Example (optional step)

> Edit argocd configmap yaml file `kubectl edit cm argocd-cm -n argocd`

> Add the `data: timeout-reconciliation` section and make it look like this

```
apiVersion: v1
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ConfigMap","metadata":{"annotations":{},"label
creationTimestamp: "2024-08-26T21:20:54Z"
labels:
  app.kubernetes.io/name: argocd-cm
  app.kubernetes.io/part-of: argocd
name: argocd-cm
namespace: argocd
resourceVersion: "7318"
uid: 6115ff37-28f1-4c0b-a477-b43f827cbb94
data:
  timeout.reconciliation: 10s
```

```
azureuser@votingAppVM2:~$ kubectl get cm -n argocd
NAME          DATA  AGE
argocd-cm      1      2d4h
argocd-cmd-params-cm  0      2d4h
argocd-gpg-keys-cm   0      2d4h
argocd-notifications-cm  0      2d4h
argocd-rbac-cm       0      2d4h
argocd-ssh-known-hosts-cm  1      2d4h
argocd-tls-certs-cm  0      2d4h
kube-root-ca.crt     1      2d4h
azureuser@votingAppVM2:~$ kubectl edit configmap argocd-cm -n argocd
```

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  timeout.reconciliation: 10s
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ConfigMap","metadata":{"annotations":{},"labels":{"app.kubernetes.io/name":"argocd-cm","app.kubernetes.io/part-of":"argocd"},"name":"argocd-cm","namespace":"argocd"}}
    creationTimestamp: "2024-08-26T21:20:54Z"
  labels:
    app.kubernetes.io/name: argocd-cm
    app.kubernetes.io/part-of: argocd
  name: argocd-cm
  namespace: argocd
  resourceVersion: "98585"
  uid: 6115ff37-28f1-4c0b-a477-b43f827cbb94
```

Note: For `data.reconciliation` in production using `10s` is not ideal; the recommended is atleast `180s` which is `3mins` this is to enable ArgoCD have enough time to update and sync without giving too much pressure to the services.

c. > View the App

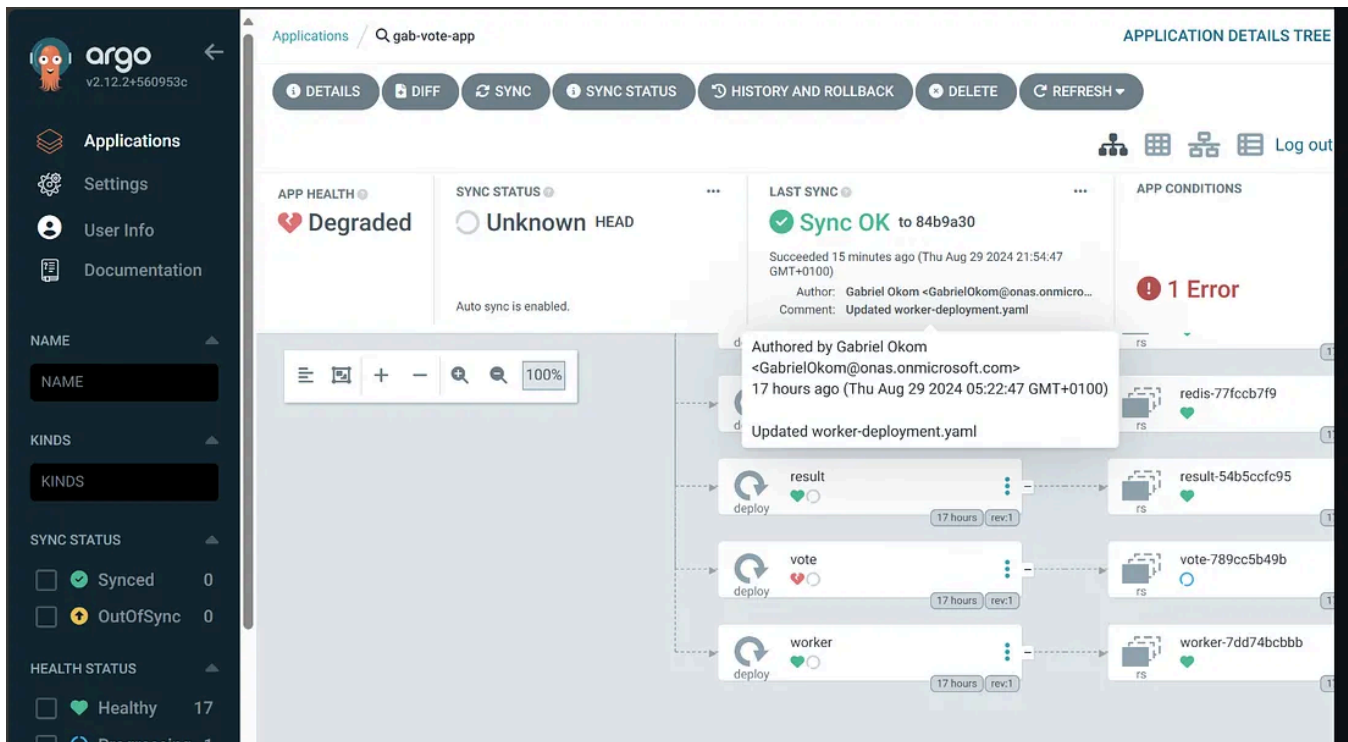
```
azureuser@votingAppVM2:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-6d9f87bb9b-m5fv6                1/1     Running   0           27h
redis-77fccb7f9-z7l6j               1/1     Running   0           27h
result-54b5ccfc95-rf6sv             1/1     Running   0           27h
vote-54799bfb67-jxcjz               0/1     ImagePullBackOff  0           27h
vote-5655bd759-dcxfb               1/1     Running   0           27h
worker-7dd74bcbbb-zb5vz             1/1     Running   0           27h
azureuser@votingAppVM2:~$
```

Kubernetes will run into an error pulling the Image from the private repository. When you run the command `kubectl get pods` > to see if all the pods are running or run the command to describe the errors `kubectl describe pod <pod name>` We can see the `ImagePullBackOff` error message more detailed

```
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type     Reason             Age          From          Message
  ----     -
Warning    FailedScheduling   16h          default-scheduler  0/1 nodes are available: 1 node(s) were unschedul
Normal     NotTriggerScaleUp  16h          cluster-autoscaler  pod didn't trigger scale-up:
Normal     NotTriggerScaleUp  19m          cluster-autoscaler  pod didn't trigger scale-up:
Warning    FailedScheduling   19m (x5 over 19m)  default-scheduler  no nodes available to schedule pods
Warning    FailedScheduling   18m          default-scheduler  0/1 nodes are available: 1 node(s) had untolerate
Normal     Scheduled          17m          default-scheduler  Successfully assigned default/vote-789cc5b49b-zxg
Normal     Pulling            15m (x4 over 17m)  kubelet          Pulling image "gabvotingappacr.azurecr.io/votinga
Warning    Failed             15m (x4 over 17m)  kubelet          Failed to pull image "gabvotingappacr.azurecr.io/
Warning    Failed             15m (x4 over 17m)  kubelet          Error: ErrImagePull
Warning    Failed             15m (x6 over 17m)  kubelet          Error: ImagePullBackOff
Normal     BackOff            2m27s (x63 over 17m)  kubelet          Back-off pulling image "gabvotingappacr.azurecr.i
```

When we check the ArgoCD `voteapp` details we can see the error too.



Step 6: Create an ACR imagePullSecret on AKS

We can not be able to view the changes made to the `app` because the Kubernetes `vote app pod` will fail with error `ImagePullBackOff`, it means that Kubernetes tried to pull the image from the container registry but failed, and is now backing off (waiting before retrying) due to repeated failures.

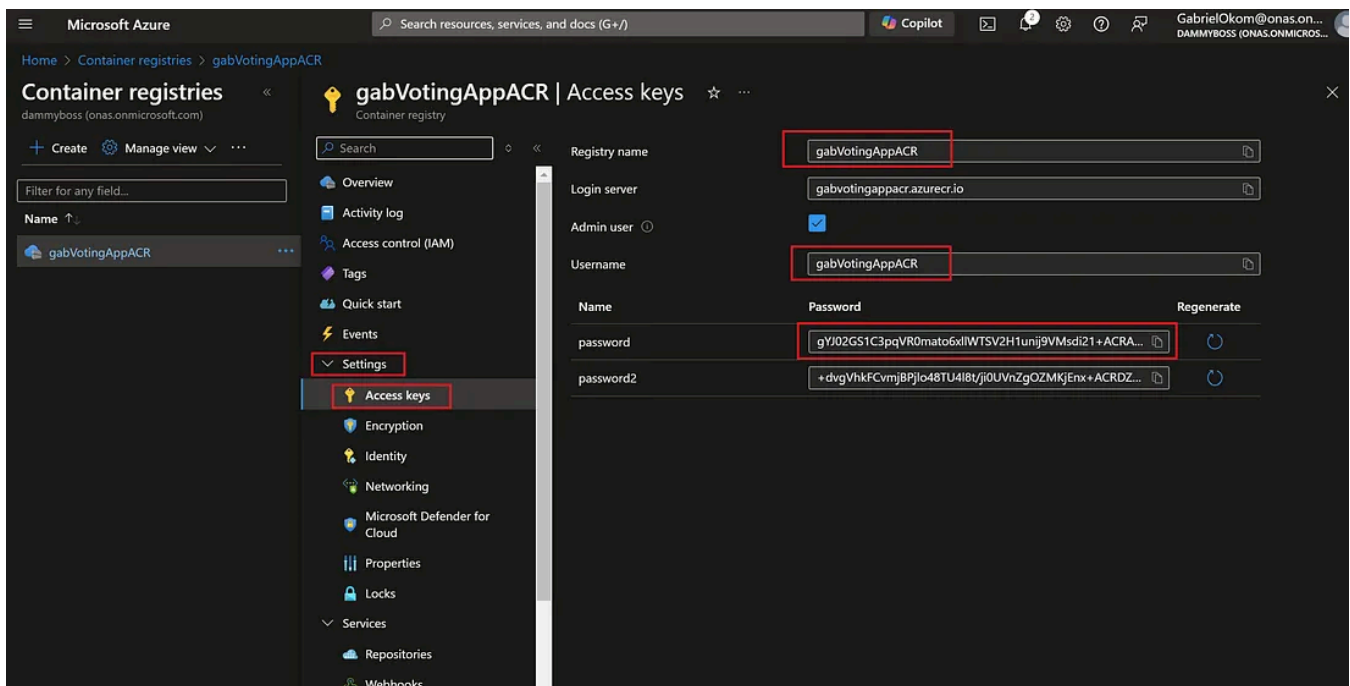
```

azureuser@votingAppVM2:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-6d9f87bb9b-m5fv6                1/1     Running   0           27h
redis-77fccb7f9-z7l6j               1/1     Running   0           27h
result-54b5ccfc95-rf6sv             1/1     Running   0           27h
vote-54799bfb67-jxcjz              0/1     ImagePullBackOff   0           27h
vote-5655bd759-dcxfb               1/1     Running   0           27h
worker-7dd74bcbbb-zb5vz             1/1     Running   0           27h
azureuser@votingAppVM2:~$

```

To correct this we need to configure `ImagePullSecret` to point where kubernetes should pull the image from.

> In your azure portal, click on the container registry > in the menu, click on settings > click on Access Keys > check Admin User > copy the password



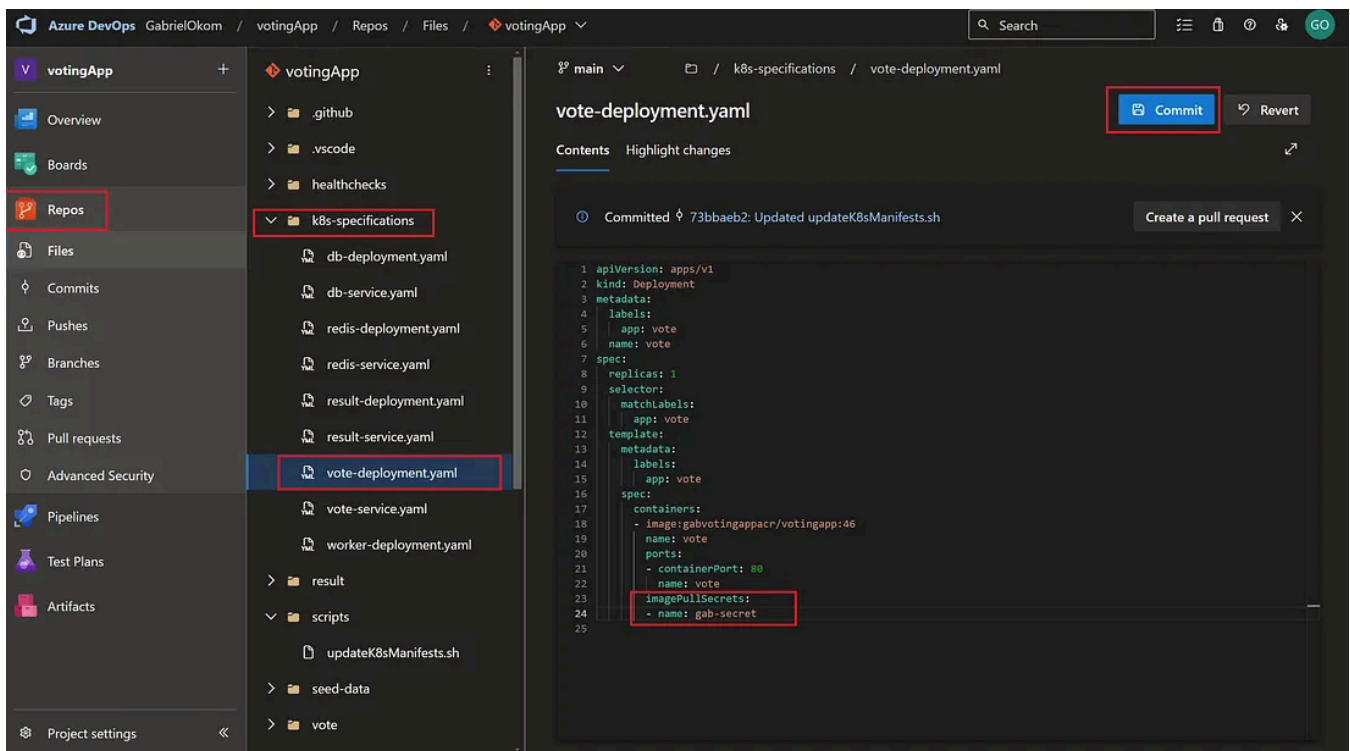
> Get a name for your secret, insert the details requested by the command and run it

```
kubectl create secret docker-registry <secret-name> \
  --namespace <namespace> \
  --docker-server=<container-registry-name>.azurecr.io \
  --docker-username=<service-principal-ID> \
  --docker-password=<service-principal-password>
```

```
azureuser@votingAppVM2:~$ kubectl create secret docker-registry gab-secret \
  --namespace default \
  --docker-server=gabVotingAppACR.azurecr.io \
  --docker-username=gabVotingAppACR \
  --docker-password=gYJ02GS1C3pqVR0mato6xllWTSV2H1unij9VMsdi21+ACRAK0Mba
secret/gab-secret created
azureuser@votingAppVM2:~$ kubectl get secret
NAME          TYPE          DATA   AGE
gab-secret    kubernetes.io/dockerconfigjson  1       69s
azureuser@votingAppVM2:~$
```

> Edit the vote deployment file and add the newly created ACR Secret

in the azure devops portal > go the repo and click on the k8s-specifications dir > edit the vote-deployment.yaml and add imagePullSecrets section inside of it with the name of the secret created earlier. > Commit the changes



run the command `kubectl get svc` to get the port number > `kubectl get pods` to see if all the pods are running > to access the page use the same node IP used for the argocd but this time you will be accessing the voteapp using the vote pod port 31000 (make sure to add/open this port number in your `vmss` inbound rule. just as we did for the argocd port number)

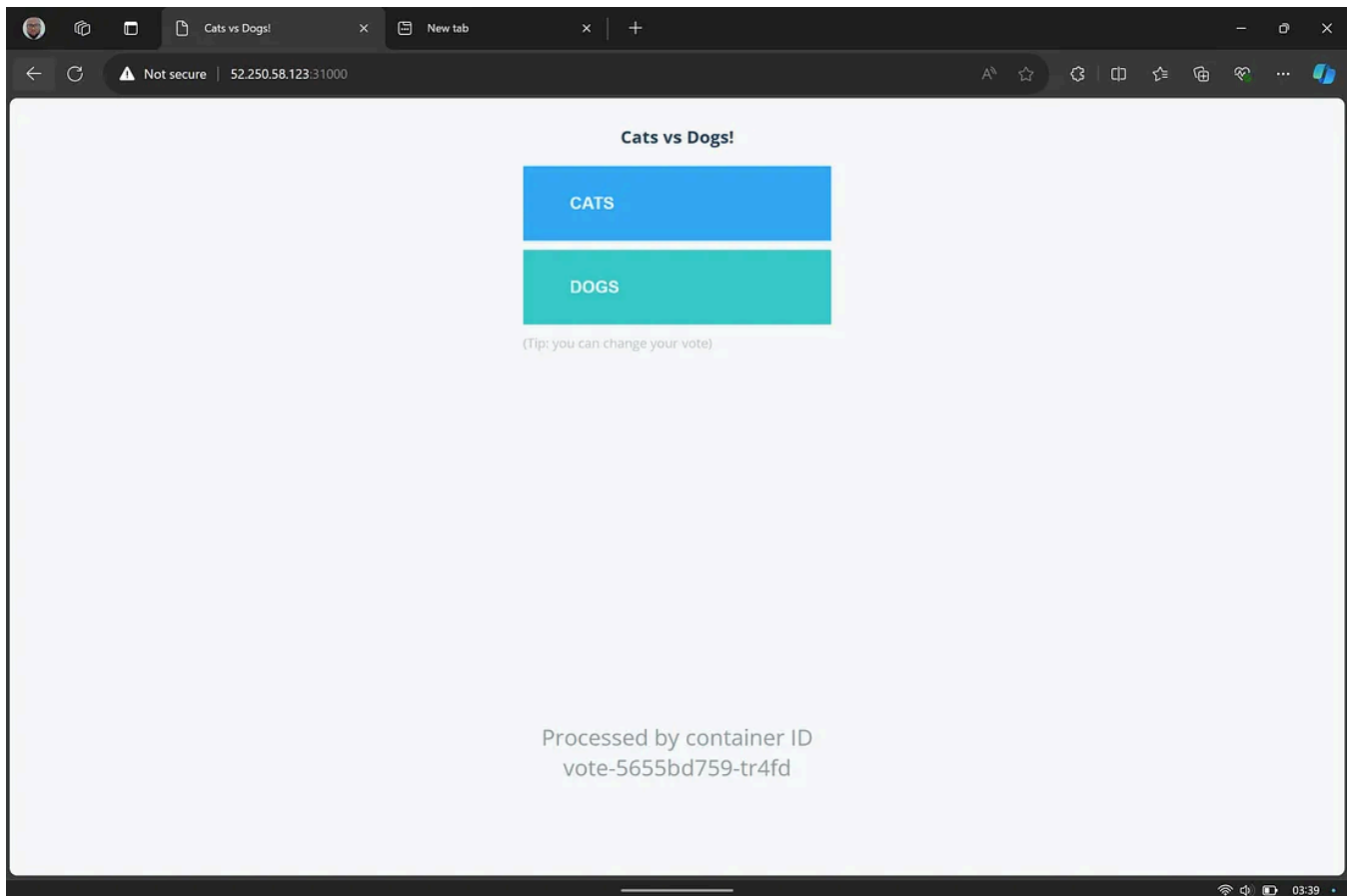
```

azureuser@votingAppVM2:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-6d9f87bb9b-vnms8                1/1     Running   0           178m
redis-77fccb7f9-lnr88              1/1     Running   0           178m
result-54b5ccfc95-rj4l8            1/1     Running   0           178m
vote-5655bd759-tr4fd               1/1     Running   0           178m
worker-7dd74bcbbb-z9pkl            1/1     Running   0           178m
azureuser@votingAppVM2:~$ kubectl get service
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
db        ClusterIP   10.0.10.34      <none>        5432/TCP         3h3m
kubernetes ClusterIP   10.0.0.1        <none>        443/TCP          12h
redis     ClusterIP   10.0.193.121    <none>        6379/TCP         3h3m
result    NodePort    10.0.190.63     <none>        5001:31001/TCP   3h3m
vote      NodePort    10.0.46.177     <none>        5000:31000/TCP   3h3m

```

view the app using `http://<nodeip-address>:31000`

(note: your port number maybe different than mine, do well to replace it)



Step 7: Verify the CICD process

In the `vote` dir in the Azure Repo, edit the `os.getenv` section of the `app.py` file. You can change the votes to something like (summer , winter) (Rain , Snow) (White , Black) etc. choose anything of your choice and commit the changes.

```
from flask import Flask, render_template, request, make_response, g
from redis import Redis
import os
import socket
import random
import json
import logging
option_a = os.getenv('OPTION_A', "Rain")
option_b = os.getenv('OPTION_B', "Snow")
hostname = socket.gethostname()
app = Flask(__name__)
unicorn_error_logger = logging.getLogger('unicorn.error')
app.logger.handlers.extend(unicorn_error_logger.handlers)
app.logger.setLevel(logging.INFO)
def get_redis():
    if not hasattr(g, 'redis'):
        g.redis = Redis(host="redis", db=0, socket_timeout=5)
    return g.redis
@app.route("/", methods=['POST','GET'])
def hello():
```

```

voter_id = request.cookies.get('voter_id')
if not voter_id:
    voter_id = hex(random.getrandbits(64))[2:-1]
vote = None
if request.method == 'POST':
    redis = get_redis()
    vote = request.form['vote']
    app.logger.info('Received vote for %s', vote)
    data = json.dumps({'voter_id': voter_id, 'vote': vote})
    redis.rpush('votes', data)
resp = make_response(render_template(
    'index.html',
    option_a=option_a,
    option_b=option_b,
    hostname=hostname,
    vote=vote,
))
resp.set_cookie('voter_id', voter_id)
return resp

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True, threaded=True)

```

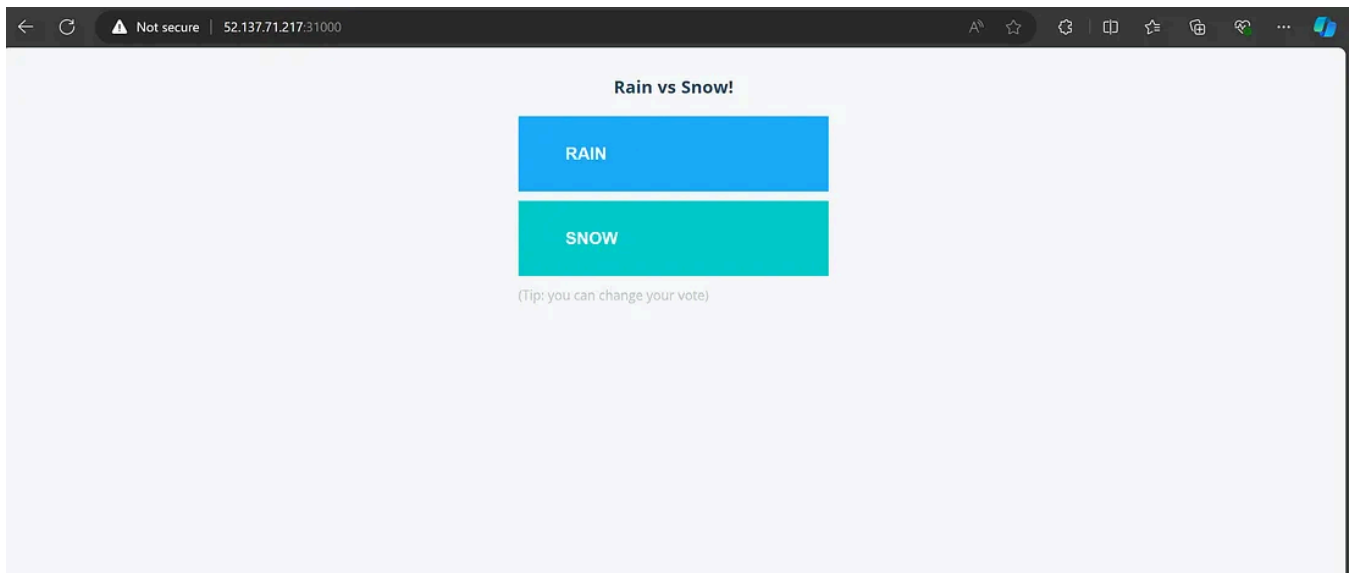
When this is done, the pipeline should trigger automatically

The screenshot shows the Azure DevOps Pipelines interface for a project named 'votingApp'. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Library, Test Plans, and Artifacts. The main area displays a table of 'Recently run pipelines'.

Pipeline	Last run
votingApp (3)	#20240902.12 • Updated app.py Individual CI for main
votingApp (2)	#20240821.1 • Update azure-pipelines-1.yml for worker microservice Manually triggered for main
votingApp	#20240821.3 • Update azure-pipelines.yml for result microservices Manually triggered for main

The first pipeline, 'votingApp (3)', is highlighted with a red box. It shows a status of 'Just now' and a duration of '27s'.

> wait for the pipeline to finish running and let's verify the app is up and running, and also verify that the changes made to the app source code has been replicated in the app UI.



This is the end of this project

However; if you have followed carefully you must have noticed we have not set up the `updateK8sManifest.sh` stage pipeline for the `worker` and `result`. Do the same for this microservices just as we did for the `vote` pipeline script to have a full functioning app.

Add the `updateK8sManifest.sh` to `worker` and `result` pipelines

When you run the pipeline, just as we did for the `vote-deployment.yaml` it is important to update the `result-deployment.yaml` and `worker-deployment.yaml` files as well to include the `imagePullSecrets`. If this is not done, you will get the same error.

```

azureuser@votingAppVM2:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-6d9f87bb9b-295bs                1/1     Running   0           46h
redis-77fccb7f9-28f4z              1/1     Running   0           46h
result-54b5ccfc95-964nd            1/1     Running   0           46h
result-9f9578dd9-wjkmq             0/1     ImagePullBackOff  0           3m33s
vote-84c5bb7ff-q2q5p              1/1     Running   0           7m40s
worker-55c7f4fc4-9q2wc             0/1     ImagePullBackOff  0           5m2s
worker-7dd74bcbbb-js9jc            1/1     Running   0           46h
azureuser@votingAppVM2:~$
azureuser@votingAppVM2:~$

```

Make sure all pods are running

```

azureuser@votingAppVM2:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
db-6d9f87bb9b-295bs                1/1     Running   0           46h
redis-77fccb7f9-28f4z              1/1     Running   0           46h
result-7fb9995d5f-7299l           1/1     Running   0           64s
vote-84c5bb7ff-q2q5p              1/1     Running   0           10m
worker-857bf8474-hrgkl            1/1     Running   0           44s
azureuser@votingAppVM2:~$

```

Worker Pipeline Script

```
trigger:
  paths:
    include:
      - worker/*

resources:
- repo: self

variables:
  dockerRegistryServiceConnection: 'ad3eca0a-4219-4a32-9df0-29fd9ba340b8'
  imageRepository: 'votingapp'
  containerRegistry: 'gabvotingappacr.azurecr.io'
  dockerfilePath: '$(Build.SourcesDirectory)/worker/Dockerfile'
  tag: '$(Build.BuildId)'

pool:
  name: 'voting-agent-app'

stages:
- stage: Build
  displayName: Build the voting App
  jobs:
  - job: Build
    displayName: Build

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(dockerRegistryServiceConnection)'
        repository: '$(imageRepository)'
        command: 'build'
        Dockerfile: 'worker/Dockerfile'
        tags: '$(tag)'

- stage: Push
  displayName: Push the voting App
  jobs:
  - job: Push
    displayName: Push

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(dockerRegistryServiceConnection)'
        repository: '$(imageRepository)'
        command: 'push'
        tags: '$(tag)'
```

```
- stage: Update_bash_script
  displayName: Update Bash Script
  jobs:
  - job: Updating_repo_with_bash
    displayName: Updating repo using bash script
    steps:
    - script: |
        # Convert line endings to Unix format
        dos2unix scripts/updateK8sManifests.sh

        # Run the shell script with the appropriate arguments
        bash scripts/updateK8sManifests.sh "worker" "${imageRepository}" "${tag}"
    displayName: Run UpdateK8sManifests Script
```

Result Pipeline Script

```
trigger:
  paths:
    include:
      - result/*

resources:
- repo: self

variables:
  dockerRegistryServiceConnection: 'ad3eca0a-4219-4a32-9df0-29fd9ba340b8'
  imageRepository: 'votingapp'
  containerRegistry: 'gabvotingappacr.azurecr.io'
  dockerfilePath: '$(Build.SourcesDirectory)/result/Dockerfile'
  tag: '$(Build.BuildId)'

pool:
  name: 'voting-agent-app'

stages:
- stage: Build
  displayName: Build the voting App
  jobs:
  - job: Build
    displayName: Build

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(dockerRegistryServiceConnection)'
        repository: '$(imageRepository)'
```

```

command: 'build'
Dockerfile: 'result/Dockerfile'
tags: '$(tag)'

- stage: Push
  displayName: Push the voting App
  jobs:
  - job: Push
    displayName: Push

    steps:
    - task: Docker@2
      inputs:
        containerRegistry: '$(dockerRegistryServiceConnection)'
        repository: '$(imageRepository)'
        command: 'push'
        tags: '$(tag)'

- stage: Update_bash_script
  displayName: Update Bash Script
  jobs:
  - job: Updating_repo_with_bash
    displayName: Updating repo using bash script
    steps:
    - script: |
        # Convert line endings to Unix format
        dos2unix scripts/updateK8sManifests.sh

        # Run the shell script with the appropriate arguments
        bash scripts/updateK8sManifests.sh "result" "$(imageRepository)" "$(tag)"
    displayName: Run UpdateK8sManifests Script

```

To check the votes, run the following commands . We can check for the votes count in either redis or db .

```

azureuser@votingAppVM2:~$ kubectl exec -it redis-77fccb7f9-28f4z -- redis-cli LRANGE votes 0 -1
(empty array)
azureuser@votingAppVM2:~$ kubectl exec -it db-6d9f87bb9b-295bs -- psql -U postgres -d postgres
psql (15.8)
Type "help" for help.

postgres=# \dt
          List of relations
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | votes | table | postgres
(1 row)

postgres=# SELECT * FROM votes;
   id   | vote
-----+-----
501a50afabe0fb9 | a
(1 row)

postgres=#

```


#devopsproject #cicdproject #cicd #devopsengineer #GabrielOkom #argocd #gitops
#AKS #Kubernetes

References:

[Abhishek.Veeramalla](#)

DevOps

Cicd

Azuredevops

Devops Project

Microservices



Follow



Written by GABRIEL OKOM

297 followers · 0 following

MSc Cyber Security and Computer Forensics | Certified DevOps Engineer

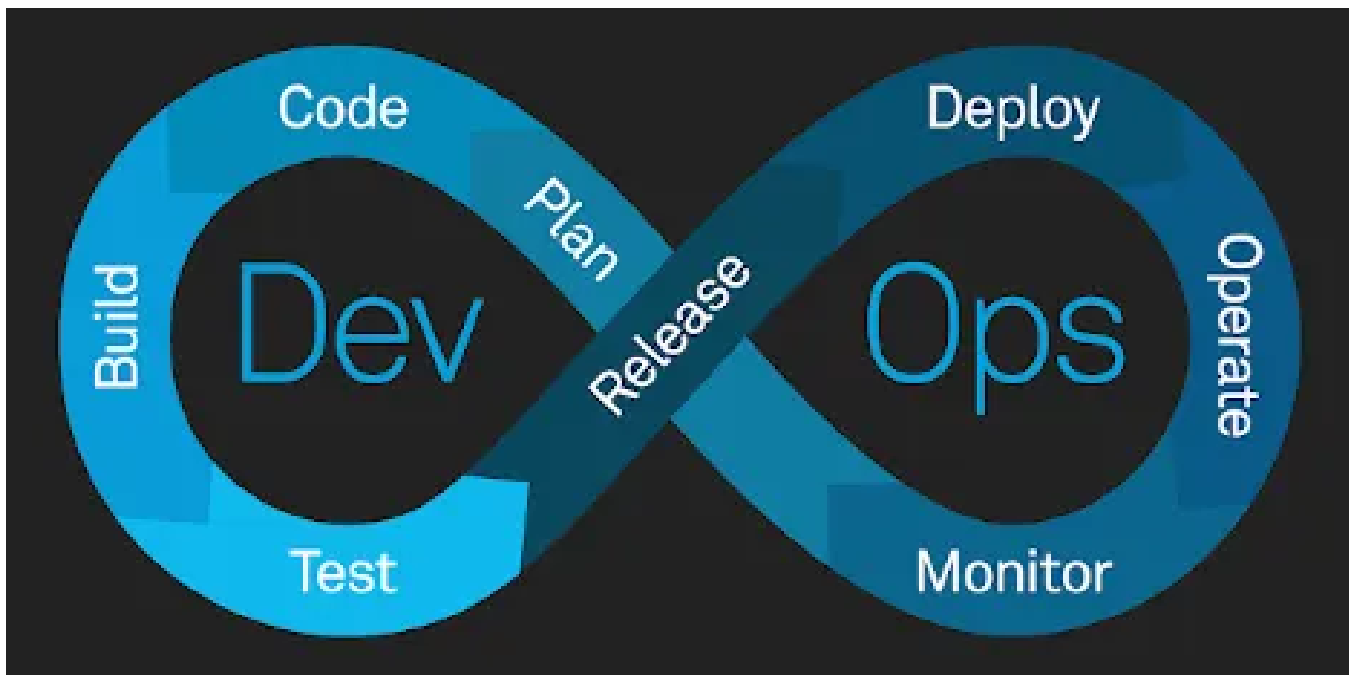
No responses yet



Write a response

What are your thoughts?

More from GABRIEL OKOM

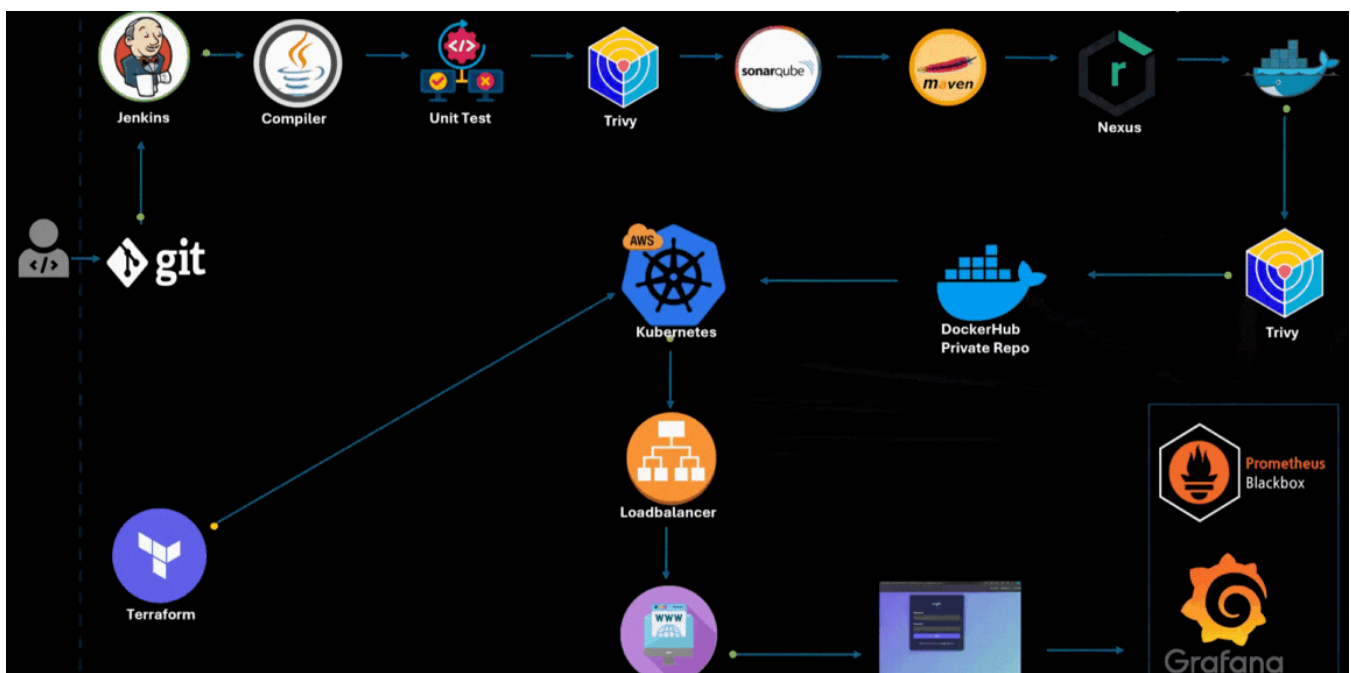


 GABRIEL OKOM

Deploy a .NET Application using Azure CI/CD Pipeline and

Step-by-step guide on how to deploy an app in Azure using app services, azure devops and github.

Nov 28, 2023  17  2



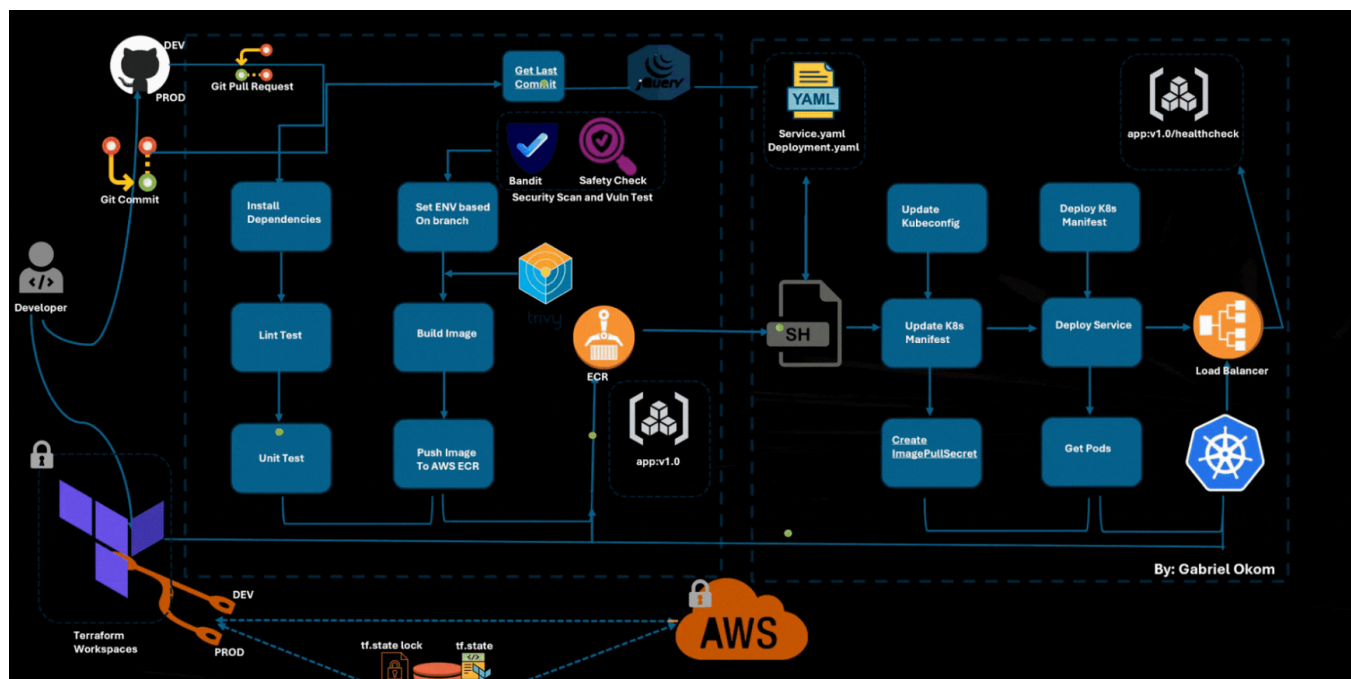


GABRIEL OKOM

CICD PROJECT: Production Level Blog APP Deployment using EKS, Nexus, SonarQube, Trivy with...

Tools Used: Jenkins: For managing the CI/CD pipeline. SonarQube: For static code analysis. Nexus: For managing dependencies and artifacts...

Sep 19, 2024 140 3



GABRIEL OKOM

CI/CD Project: Deploy a Python App with /healthcheck Endpoint on Docker, ECR, Kubernetes...

Table of Contents

Nov 4, 2024 32





GABRIEL OKOM

How to Configure Self-Hosted Runners for your CI/CD Pipelines using GitActions

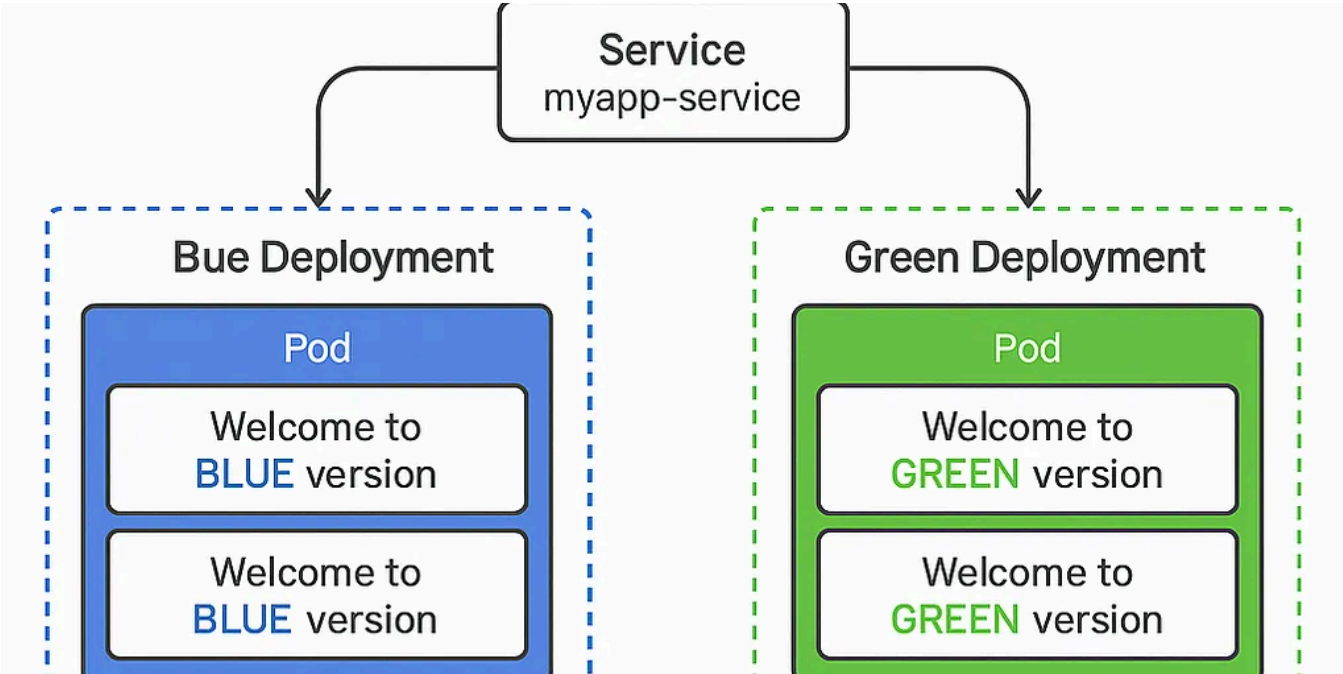
Setting Up Self-Hosted Runners with GitHub Actions.

May 16, 2024 🖱 4



See all from GABRIEL OKOM

Recommended from Medium



 In AWS in Plain English by Aman Pathak | DevOps | AWS | K8s | Terraform

Blue/Green Deployment in 5 Minutes- Kubernetes Deployment Strategy Hands-On

 Please Read

 Jun 28  8



Component	Name
Resource Group	devops
AKS Cluster	aks
Azure Container Registry	poojareddy
Azure Key Vault	poojavault
Azure DevOps Repo	devops-training
Helm chart + pipeline	In repo

 Pooja Bolla

End-to-End CI/CD Pipeline on Azure: AKS + ACR + Key Vault + Azure DevOps

“From code commit to containerized deployment — harness the full power of Azure DevOps, AKS, ACR, and Key Vault in one streamlined...

★ May 2 🖱 2



👤 Neamul Kabir Emon

Build a Real-World DevOps Pipeline with GitHub Actions, Terraform, EKS, and Argo CD (Step-by-Step)

Introduction

Apr 16 🖱 50 💬 1



Пельменная №2

Каталог

Пельмени
\$5
С говядиной

Добавить в корзину

Хинкали
\$3.5
Со свининой

Добавить в корзину

Манты
\$2.75
С мясом молодых бычков

Добавить в корзину

Корзина

Корзина пуста

Итого: \$0

Оформить

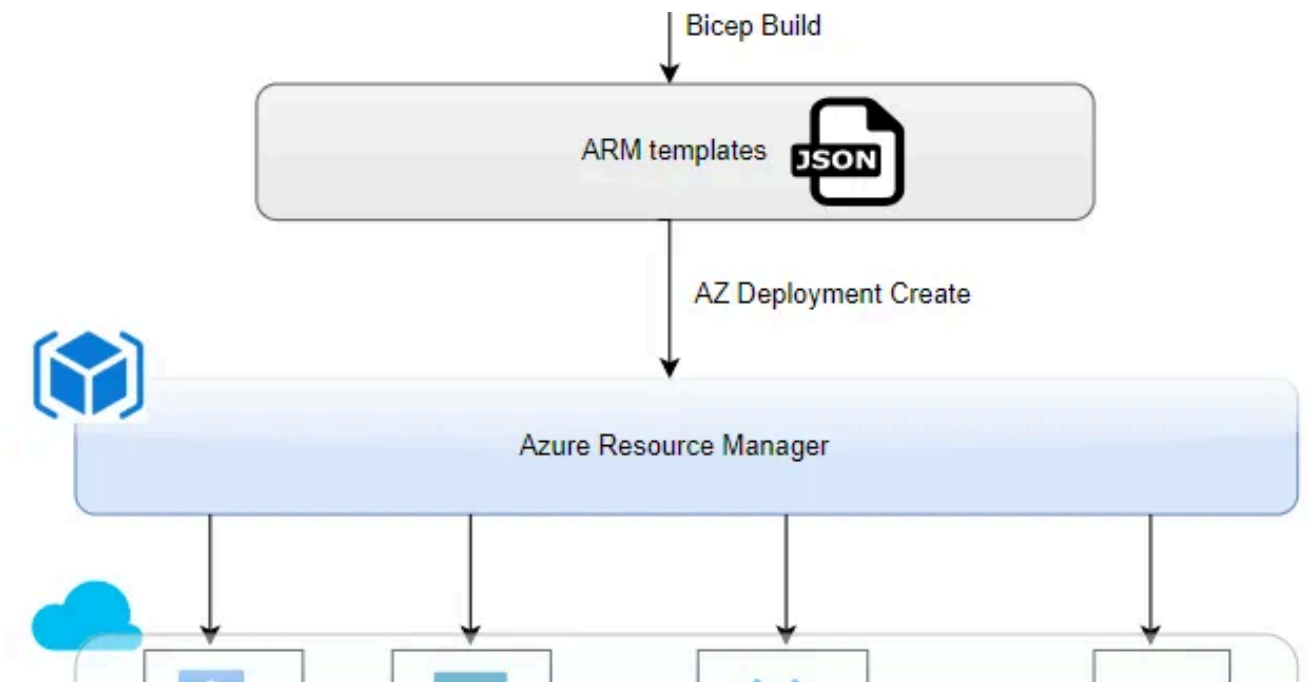


Kanat Akylson

From Zero to DevOps Hero: How I Built a Full CI/CD Pipeline with Terraform, Kubernetes, and ArgoCD

Ever wondered how to take a simple app and deploy it like a pro with full automation? In this step-by-step guide, I'll share how I went...

★ May 14 🖱 17



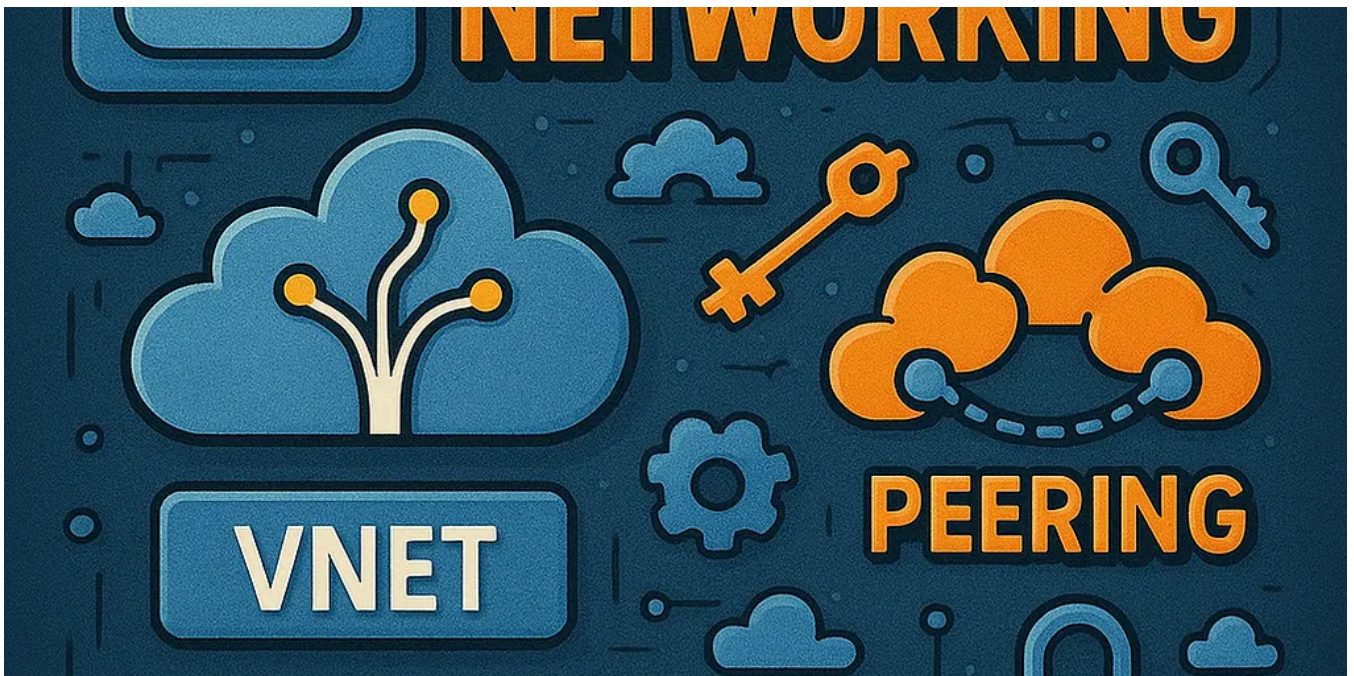
Sridhar

Automating Azure Deployments with Bicep and Azure Pipelines

Azure IaC Best Practices: Automating Infrastructure with Bicep and Azure Pipelines

★ May 17 🖱 9





 Vinoth Subbiah

Day 5: Azure Networking—VNet, NSG, Peering, Private Link

 Networking is the backbone of Azure. Today, we'll cover how to design secure, scalable networks using Azure Virtual Networks (VNETs)...

✦ Jun 1 🖱 25



See more recommendations